Java

Tricky Interview Questions & Answers

100 Expert-Level Questions with Code Examples

• Java Fundamentals & Tricky Output: i++, String ==, integer cache, float precision
• OOP, Inheritance & Polymorphism: method hiding, covariant returns, equals/hashCode
• Generics, Collections & Lambdas: type erasure, PECS, streams, effectively final
• Exception Handling: finally return, try-with-resources, suppressed exceptions
• Concurrency Traps: visibility, volatile, ThreadLocal leaks, double-checked locking
• JVM Internals: memory areas, GC generations, reference types, class loading
• String, StringBuilder & Interning: pool, compact strings, concatenation tricks
• Modern Java (8-21): Optional, Records, Sealed Classes, Virtual Threads, Text Blocks
• Design Patterns & Gotchas: static init order, defensive copies, Cloneable pitfalls
• Expert Edge Cases: NaN, subList views, varargs generics, happens-before, diamond

Java 8 • Java 11 • Java 17 • Java 21

SECTION 1 — Java Fundamentals & Tricky Output

Q1. What does this print? int i = 0; System.out.println(i++ + ++i); [Tricky] [Output]

Answer

It prints 2. Evaluation is left-to-right: i++ returns 0 and increments i to 1; then ++i increments i to 2 and returns 2. So 0 + 2 = 2. The post-increment and pre-increment operators interact with the expression evaluation order in a non-obvious way. This is a classic tricky Java interview output question.

Example

```
int i = 0;
System.out.println(i++ + ++i);  // prints 2
// Step 1: i++ -> returns 0, i becomes 1
// Step 2: ++i -> i becomes 2, returns 2
// Step 3: 0 + 2 = 2

int j = 5;
System.out.println(j-- - --j);  // prints 3
// Step 1: j-- -> returns 5, j becomes 4
// Step 2: --j -> j becomes 3, returns 3
// Step 3: 5 - 3 = 3
```

Q2. Why does "==" sometimes work for String comparison and sometimes fail? [Tricky] [String]

Answer

String literals are interned — placed in the String pool — so two literals with the same content share the same object reference, making == return true. String objects created with new are NOT interned by default, so == compares references (different objects), returning false. Always use .equals() for content comparison.

Example

```
String a = "hello";
String b = "hello";
System.out.println(a == b);       // true  (both from string pool)
System.out.println(a.equals(b));  // true

String c = new String("hello");
String d = new String("hello");
System.out.println(c == d);       // false (different objects on heap)
System.out.println(c.equals(d)); // true  (same content)

// Force interning
String e = c.intern();
System.out.println(a == e);       // true  (e now points to pool)

// Rule: always use .equals() for String comparison
```

Q3. What does this print? System.out.println(1 + 2 + "3" + 4 + 5); [Tricky] [Output]

Answer

It prints "3345". The + operator is left-associative. First 1+2=3 (int + int = int), then 3+"3"="33" (int + String = String concatenation), then "33"+4="334", then "334"+5="3345". This demonstrates the importance of operator associativity and how String concatenation interacts with arithmetic.

Example

```
System.out.println(1 + 2 + "3" + 4 + 5);   // "3345"
```

```
// 1+2=3 -> 3+"3"="33" -> "33"+4="334" -> "334"+5="3345"

System.out.println("1" + 2 + 3);             // "123"
// "1"+2="12" -> "12"+3="123"

System.out.println("1" + (2 + 3));           // "15"
// 2+3=5 first (parentheses), then "1"+5="15"

System.out.println(1 + 2 + 3 + "end");      // "6end"
// 1+2=3, 3+3=6, 6+"end"="6end"
```

Q4. Why does Integer.valueOf(127) == Integer.valueOf(127) return true but Integer.valueOf(128) == Integer.valueOf(128) return false? [Tricky] [Integer Cache]

Answer

Java caches Integer objects for values -128 to 127 (the Integer cache). valueOf() returns the cached instance for values in this range, so == compares the same object reference. For values outside this range, valueOf() creates new objects each time, so == compares different references. This is a famous Java gotcha.

Example

```
Integer a = Integer.valueOf(127);
Integer b = Integer.valueOf(127);
System.out.println(a == b);       // true  (same cached object)

Integer c = Integer.valueOf(128);
Integer d = Integer.valueOf(128);
System.out.println(c == d);       // false (different objects)

// Autoboxing uses Integer.valueOf() internally
Integer x = 127;   // same as Integer.valueOf(127)
Integer y = 127;
System.out.println(x == y);   // true

Integer p = 128;
Integer q = 128;
System.out.println(p == q);   // false!

// ALWAYS use .equals() for Integer comparison
System.out.println(p.equals(q)); // true
```

Q5. What is the output? byte b = (byte) 130; System.out.println(b); [Tricky] [Type Casting]

Answer

It prints -126. byte is an 8-bit signed type with range -128 to 127. 130 in binary is 10000010. When cast to byte, the top bits are truncated. The most significant bit is 1, making it negative in two's complement. 130 - 256 = -126. This is narrowing conversion overflow — a common Java pitfall.

Example

```
byte b = (byte) 130;
System.out.println(b);  // -126
// 130 = 10000010 in binary
// As signed byte: 10000010 = -(256-130) = -126

byte b2 = (byte) 255;
System.out.println(b2); // -1
// 255 = 11111111 -> two's complement = -1

byte b3 = (byte) 256;
```

```
System.out.println(b3); // 0
// 256 = 100000000 -> truncate to 8 bits = 00000000 = 0

// Overflow formula: (value % 256) if in signed range
// else subtract 256
int i = (byte) 200;
System.out.println(i);  // -56  (200 - 256 = -56)
```

Q6. What happens when you mix primitives and wrappers in == comparison? [Tricky] [Autoboxing]

Answer

When a primitive is compared to a wrapper using ==, the wrapper is unboxed to a primitive and the comparison is value-based. This can cause NullPointerException if the wrapper is null. It also means comparing int and Integer with == is always a value comparison, unlike comparing two Integers with ==.

Example

```
int i = 100;
Integer boxed = 100;
System.out.println(i == boxed);  // true (boxed unboxed to int)

Integer a = 200;
Integer b = 200;
System.out.println(a == b);      // false (reference comparison)
System.out.println(a == 200);    // true  (b unboxed to int)

// NullPointerException trap
Integer nullInt = null;
try {
    System.out.println(i == nullInt); // NPE! unboxing null
} catch (NullPointerException e) {
    System.out.println("NPE on null unboxing");
}
```

Q7. What is the output of a switch statement with fall-through? [Tricky] [Switch]

Answer

Without a break statement, Java switch falls through to the next case. This is a frequent source of bugs. Each case block executes until it encounters a break or the switch ends. Java 14+ switch expressions eliminate fall-through with arrow syntax. Always use break or return in traditional switch statements to avoid accidental fall-through.

Example

```
int x = 2;
switch (x) {
    case 1: System.out.println("one");
    case 2: System.out.println("two");  // NO break
    case 3: System.out.println("three");// NO break
    case 4: System.out.println("four"); break;
    default: System.out.println("other");
}
// Prints: two, three, four  (falls through!)

// Java 14+ switch expression: no fall-through
String result = switch (x) {
    case 1 -> "one";
    case 2 -> "two";   // no fall-through possible
    case 3 -> "three";
    default -> "other";
};
```

```
System.out.println(result); // "two"
```

Q8. What is the output? System.out.println(0.1 + 0.2 == 0.3); [Tricky] [Output]

Answer

It prints false. Floating-point numbers are represented in binary IEEE 754 format. 0.1 and 0.2 cannot be represented exactly in binary, so 0.1 + 0.2 = 0.30000000000000004, not exactly 0.3. Never use == to compare floating-point numbers. Use a tolerance (epsilon) comparison or BigDecimal for exact decimal arithmetic.

Example

```
System.out.println(0.1 + 0.2);          // 0.30000000000000004
System.out.println(0.1 + 0.2 == 0.3);   // false

// CORRECT: use epsilon comparison
double epsilon = 1e-10;
boolean equal = Math.abs((0.1 + 0.2) - 0.3) < epsilon;
System.out.println(equal);               // true

// CORRECT: use BigDecimal for exact arithmetic
import java.math.BigDecimal;
BigDecimal bd1 = new BigDecimal("0.1");  // use String constructor!
BigDecimal bd2 = new BigDecimal("0.2");
BigDecimal sum  = bd1.add(bd2);
System.out.println(sum);                 // 0.3
System.out.println(sum.compareTo(new BigDecimal("0.3")) == 0); // true

// WRONG: BigDecimal(double) has same FP issue
// new BigDecimal(0.1) -> 0.1000000000000000055511...
```

Q9. Can you call a static method on a null reference? What happens? [Tricky] [Null]

Answer

Yes — calling a static method on a null reference does NOT throw a NullPointerException. Static methods are resolved at compile time based on the declared type, not the runtime object. The compiler converts the call to a direct static method invocation, ignoring the null reference entirely. This is counterintuitive and a common interview trick.

Example

```
class Greeter {
    static String hello() { return "Hello!"; }
    String name() { return "World"; }
}

Greeter g = null;
System.out.println(g.hello()); // "Hello!" — NO NPE!
// Compiler translates to: Greeter.hello()

// This DOES throw NPE:
try {
    System.out.println(g.name()); // NPE: instance method on null
} catch (NullPointerException e) {
    System.out.println("NPE as expected");
}

// IntelliJ/compiler warn about this as a code smell
```

Q10. What is the output? String s = "abc"; s.toUpperCase(); System.out.println(s); [Tricky] [String]

Answer

It prints "abc" — the original string is unchanged. Strings in Java are immutable. toUpperCase() returns a NEW String with the uppercase version. The return value is discarded. This is a very common beginner trap. The fix is to assign the result: s = s.toUpperCase().

Example

```
String s = "abc";
s.toUpperCase();              // result is DISCARDED
System.out.println(s);        // "abc" — unchanged!

// CORRECT: capture the return value
s = s.toUpperCase();
System.out.println(s);        // "ABC"

// Same trap with all String methods:
String t = "  hello  ";
t.trim();                     // result discarded
System.out.println(t);        // "  hello  " — unchanged!

t = t.trim();                 // correct
System.out.println(t);        // "hello"

// String is IMMUTABLE: every mutation returns a new object
```

SECTION 2 — OOP, Inheritance & Polymorphism

Q11. What is the output when a subclass overrides a method called in the superclass constructor? [OOP] [Tricky]

Answer

The overridden method in the subclass is called — even before the subclass constructor runs. This is because Java uses dynamic dispatch for instance methods even in constructors. The subclass's overriding method executes when the object's instance fields are still at their default values (null, 0, false), leading to surprising results.

Example

```
class Base {
    Base() { print(); }             // calls overridden method!
    void print() { System.out.println("Base.print"); }
}

class Child extends Base {
    String name = "Child";
    Child() { super(); }
    @Override
    void print() {
        System.out.println("Child.print, name=" + name);
    }
}

new Child();
// Prints: Child.print, name=null
// name is null because Child fields not initialized yet!
```

```
// Rule: never call overridable methods from constructors
// Use private or final methods in constructors instead
```

Q12. What is the difference between method hiding and method overriding? [OOP] [Tricky]

Answer

Overriding: when a subclass redefines an instance method from the superclass — the subclass version is called based on the runtime type (dynamic dispatch). Hiding: when a subclass defines a static method with the same signature — the version called depends on the DECLARED (compile-time) type of the reference, NOT the runtime type. Static methods are never overridden, only hidden.

Example

```
class Parent {
    static void staticMethod() { System.out.println("Parent static"); }
    void instanceMethod()      { System.out.println("Parent instance"); }
}

class Child extends Parent {
    // HIDING (not overriding):
    static void staticMethod() { System.out.println("Child static"); }
    // OVERRIDING:
    @Override
    void instanceMethod()      { System.out.println("Child instance"); }
}

Parent p = new Child();
p.staticMethod();   // "Parent static" (compile-time type = Parent)
p.instanceMethod(); // "Child instance" (runtime type  = Child)

Child c = new Child();
c.staticMethod();   // "Child static"
c.instanceMethod(); // "Child instance"
```

Q13. Can a private method be overridden? What is the output? [OOP] [Gotcha]

Answer

No, private methods cannot be overridden — they are not visible to subclasses. If a subclass defines a method with the same signature as a private superclass method, it creates a new independent method. When calling via a superclass reference, the private superclass method is called (no dynamic dispatch). This can produce counterintuitive output.

Example

```
class A {
    private void display() { System.out.println("A.display"); }
    void show() { display(); } // calls A's private display
}

class B extends A {
    // This is a NEW method, not an override
    void display() { System.out.println("B.display"); }
}

A obj = new B();
obj.show();      // "A.display" (A.show calls A's private display)

// B.display is never called through A.show
// because A.show does not see B.display
```

```
B b = new B();
b.display();     // "B.display" (calling directly on B)
```

Q14. What happens when a class implements two interfaces with the same default method?

[OOP] [Interface]

Answer

A compile error occurs — the class MUST override the conflicting default method explicitly, otherwise the compiler cannot determine which version to use. Inside the overriding method, you can call a specific interface's default method using InterfaceName.super.method(). This is Java's resolution of the diamond problem for default methods.

Example

```
interface A {
    default void hello() { System.out.println("A.hello"); }
}
interface B {
    default void hello() { System.out.println("B.hello"); }
}

// COMPILE ERROR without override:
// class C implements A, B {}  // Error: inherits unrelated defaults

// CORRECT: must override
class C implements A, B {
    @Override
    public void hello() {
        A.super.hello(); // explicitly call A's version
        B.super.hello(); // explicitly call B's version
        System.out.println("C.hello");
    }
}

new C().hello();
// A.hello
// B.hello
// C.hello
```

Q15. What is covariant return types in Java? [OOP] [Tricky]

Answer

A covariant return type allows an overriding method to return a more specific (sub)type than the one declared in the superclass. Introduced in Java 5. This makes APIs cleaner — callers using the subclass reference get a more specific type without casting. The JVM handles this via bridge methods for backward compatibility.

Example

```
class Animal {
    Animal create() {
        return new Animal();
    }
}

class Dog extends Animal {
    @Override
    Dog create() {           // covariant: Dog is a subtype of Animal
        return new Dog();
    }
}
```

```
Dog dog = new Dog();
Dog d = dog.create();  // no cast needed!

// Without covariance: would need cast
// Dog d = (Dog) dog.create();

// Builder pattern uses covariant returns:
class Builder {
    Builder setName(String n) { return this; }
}
class ConcreteBuilder extends Builder {
    @Override ConcreteBuilder setName(String n) { return this; }
}
```

Q16. Can an abstract class have a constructor? Can you instantiate it? [OOP] [Abstract]

Answer

Yes, abstract classes can (and should) have constructors. They cannot be directly instantiated with new, but their constructors ARE called when a concrete subclass is instantiated via super(). The abstract class constructor initializes its own fields. Not defining a constructor in an abstract class means the default no-arg constructor is available but might leave fields uninitialized.

Example

```
abstract class Shape {
    String color;
    Shape(String color) {        // Constructor in abstract class
        this.color = color;
        System.out.println("Shape created: " + color);
    }
    abstract double area();
}

class Circle extends Shape {
    double radius;
    Circle(String color, double r) {
        super(color);            // calls abstract class constructor
        this.radius = r;
    }
    @Override double area() { return Math.PI * radius * radius; }
}

// new Shape("red"); // COMPILE ERROR: cannot instantiate abstract class
Circle c = new Circle("red", 5.0);
// Prints: Shape created: red
System.out.println(c.color); // "red"
```

Q17. What is the output? class A { int x = 10; } class B extends A { int x = 20; } A obj = new B(); System.out.println(obj.x); [OOP] [Tricky]

Answer

It prints 10. Fields are NOT polymorphic in Java — field access is resolved at compile time based on the declared (static) type, not the runtime type. This is called field hiding. Only methods are polymorphic (resolved at runtime). obj is declared as A so obj.x accesses A's x, not B's x.

Example

```
class A { int x = 10; }
class B extends A { int x = 20; }  // hides A.x, NOT overrides
```

```
A obj = new B();
System.out.println(obj.x);         // 10 (compile-time type = A)

B bObj = new B();
System.out.println(bObj.x);        // 20 (compile-time type = B)

// Accessing superclass field from subclass:
class B2 extends A {
    int x = 20;
    void printBoth() {
        System.out.println(x);       // 20 (B2.x)
        System.out.println(super.x); // 10 (A.x)
    }
}

// KEY: fields are resolved by declared type, methods by runtime type
```

Q18. What is the difference between final class, final method, and final variable? [OOP] [Final]

Answer

final class: cannot be subclassed (e.g., String, Integer). final method: cannot be overridden by subclasses (but can be inherited). final variable: can be assigned only once — must be initialized at declaration, in a constructor (for instance fields), or in a static initializer (for static fields). A final reference can still have its object mutated — only the reference itself cannot be reassigned.

Example

```
// final class: cannot extend
final class Immutable { int value; }
// class Broken extends Immutable {}  // COMPILE ERROR

// final method: cannot override
class Base {
    final void lock() { System.out.println("locked"); }
}
// class Sub extends Base {
//     void lock() {} // COMPILE ERROR
// }

// final variable: assigned once
final int MAX = 100;
// MAX = 200;  // COMPILE ERROR

// final reference: reference fixed, object mutable!
final List<String> list = new ArrayList<>();
list.add("hello");  // OK: mutating the object
// list = new ArrayList<>(); // COMPILE ERROR: reassigning reference

// final in lambda: effectively final (Java 8+)
int count = 0;
Runnable r = () -> System.out.println(count); // count is eff. final
// count++; // would break effective-final status
```

Q19. What is the output of instanceof with null? [OOP] [Tricky]

Answer

null instanceof AnyType always returns false — it never throws a NullPointerException. This is by specification. This makes instanceof safe to use without a null check first. Java 16+ pattern matching instanceof also handles null safely in the same way.

Example

```
Object obj = null;
System.out.println(obj instanceof Object);  // false
System.out.println(obj instanceof String);  // false
System.out.println(null instanceof String); // false

// No NPE! Useful for null-safe type checks
String s = null;
if (s instanceof String) {  // false, never enters block
    System.out.println("Is a String");
}

// Java 16 pattern matching with null
Object o = null;
if (o instanceof String str) {
    System.out.println("String: " + str); // not executed for null
} else {
    System.out.println("Not a String");   // prints this
}
```

Q20. What happens when equals() is overridden but hashCode() is not? [OOP] [Gotcha]

Answer

Objects that are equal by equals() must have the same hashCode() — this is the equals-hashCode contract. Breaking this contract causes HashMap and HashSet to behave incorrectly: two "equal" objects may be stored in different buckets, and HashSet may contain "duplicates". Always override hashCode() when you override equals().

Example

```
class BadKey {
    String name;
    BadKey(String n) { this.name = n; }
    @Override
    public boolean equals(Object o) {
        return o instanceof BadKey && name.equals(((BadKey)o).name);
    }
    // hashCode NOT overridden -- uses Object.hashCode() (identity)
}

Map<BadKey, String> map = new HashMap<>();
BadKey k1 = new BadKey("alice");
map.put(k1, "value");

BadKey k2 = new BadKey("alice");  // equal to k1
System.out.println(k1.equals(k2)); // true
System.out.println(map.get(k2));   // null! Different hash bucket!

// CORRECT: override both
@Override public int hashCode() {
    return Objects.hash(name);
}
```

SECTION 3 — Generics, Collections & Lambdas

Q21. What is type erasure and what are its runtime implications? [Generics] [Tricky]

▶ Answer

Generics in Java are implemented through type erasure: type parameters are erased at compile time and replaced with their bounds (or Object if unbounded). At runtime, List<String> and List<Integer> are both just List. This means: you cannot use instanceof with generic types, cannot create arrays of generic types, and cannot call new T(). The compiler adds casts where needed.

⌨ Example

```
List<String> strings = new ArrayList<>();
List<Integer> ints = new ArrayList<>();

// Same class at runtime due to type erasure
System.out.println(strings.getClass() == ints.getClass()); // true
System.out.println(strings.getClass().getName()); // java.util.ArrayList

// CANNOT do at runtime:
// if (obj instanceof List<String>) {}  // COMPILE ERROR

// CANNOT create generic array:
// T[] arr = new T[10];  // COMPILE ERROR

// Workaround: use Class<T> token
class Box<T> {
    private final Class<T> type;
    Box(Class<T> type) { this.type = type; }
    T cast(Object o) { return type.cast(o); }
}
Box<String> box = new Box<>(String.class);
```

Q22. What is the difference between List<?>, List<Object>, and List<T>? [Generics] [Tricky]

▶ Answer

List<Object>: you can add any Object, but a List<String> cannot be passed where List<Object> is expected (not covariant). List<?>: unbounded wildcard — read-only (you can only add null); accepts any List<X>. List<T>: generic type parameter — the specific type is fixed and known within the context; you can read AND write elements of type T.

⌨ Example

```
// List<Object>: NOT covariant with List<String>
List<String> strings = new ArrayList<>();
// List<Object> objs = strings; // COMPILE ERROR

// List<?>: read-only, accepts any List
List<?> wildcard = strings; // OK
// wildcard.add("hello");    // COMPILE ERROR: cannot add
Object o = wildcard.get(0);  // OK: reads as Object

// Useful for generic methods:
void printAll(List<?> list) {
    for (Object item : list) System.out.println(item);
}
printAll(strings);         // OK
printAll(new ArrayList<Integer>()); // OK

// List<T>: full type safety
<T> void copy(List<T> from, List<T> to) {
```

```
    for (T item : from) to.add(item); // can read and write T
}
```

Q23. Explain PECS: Producer Extends, Consumer Super. [Generics] [Gotcha]

▶ Answer

PECS is the rule for using bounded wildcards. If a collection is used as a producer (you read from it), use extends: List<? extends T>. If it is used as a consumer (you write to it), use super: List<? super T>. If both, use T directly. This is the principle behind Collections.copy(List<? super T>, List<? extends T>).

⌨ Example

```
// PRODUCER: read elements from a list -> use extends
double sumList(List<? extends Number> list) {
    double sum = 0;
    for (Number n : list) sum += n.doubleValue(); // read OK
    return sum;
}
sumList(new ArrayList<Integer>());  // OK
sumList(new ArrayList<Double>());   // OK

// CONSUMER: write elements into a list -> use super
void addNumbers(List<? super Integer> list) {
    list.add(1);   // write Integer OK
    list.add(2);
}
addNumbers(new ArrayList<Integer>()); // OK
addNumbers(new ArrayList<Number>());  // OK
addNumbers(new ArrayList<Object>());  // OK

// PECS in action: Collections.copy
// static <T> void copy(List<? super T> dest,
//                      List<? extends T> src)
// dest is consumer (super), src is producer (extends)
```

Q24. What is the difference between fail-fast and fail-safe iterators? [Collections] [Tricky]

▶ Answer

Fail-fast iterators (ArrayList, HashMap) throw ConcurrentModificationException if the collection is structurally modified during iteration (except through the iterator itself). They track a modCount. Fail-safe iterators (CopyOnWriteArrayList, ConcurrentHashMap) iterate over a snapshot and do not throw CME, but may not reflect recent changes.

⌨ Example

```
// FAIL-FAST: throws ConcurrentModificationException
List<String> list = new ArrayList<>(List.of("a","b","c"));
for (String s : list) {
    if (s.equals("b")) list.remove(s); // CME!
}

// CORRECT: use iterator's remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) it.remove(); // safe
}

// Or: removeIf (Java 8+)
list.removeIf(s -> s.equals("b")); // no CME

// FAIL-SAFE: CopyOnWriteArrayList
```

```
List<String> cowList = new CopyOnWriteArrayList<>(List.of("a","b","c"));
for (String s : cowList) {
    cowList.remove(s); // NO CME (iterates snapshot)
}
System.out.println(cowList); // []
```

Q25. What is the difference between Predicate.and() and && in Java lambdas? [Lambda] [Functional]

Answer

Predicate.and() composes two predicates, returning a new Predicate that is true only if both are true. It is short-circuiting: if the first predicate is false, the second is not evaluated. && in a lambda is just normal boolean AND inside the lambda body. The difference is composability: Predicate.and() allows combining predicates as objects at runtime.

Example

```
import java.util.function.Predicate;

Predicate<String> notNull  = s -> s != null;
Predicate<String> notEmpty = s -> !s.isEmpty();
Predicate<String> longStr  = s -> s.length() > 5;

// Compose with .and(): short-circuits if first is false
Predicate<String> valid = notNull.and(notEmpty).and(longStr);
System.out.println(valid.test("hello world")); // true
System.out.println(valid.test(""));             // false
System.out.println(valid.test(null));           // false (short-circuit)

// Using || equivalent: .or()
Predicate<Integer> small = n -> n < 5;
Predicate<Integer> large = n -> n > 100;
Predicate<Integer> extreme = small.or(large);
System.out.println(extreme.test(3));   // true
System.out.println(extreme.test(200)); // true
System.out.println(extreme.test(50));  // false

// Negate
Predicate<Integer> middle = extreme.negate();
System.out.println(middle.test(50)); // true
```

Q26. What is effectively final and why do lambdas require it? [Lambda] [Tricky]

Answer

A variable is "effectively final" if it is never reassigned after initialization — it behaves as if it were declared final. Lambdas (and anonymous classes) can capture local variables from their enclosing scope only if those variables are final or effectively final. This restriction exists because the lambda may outlive the method stack frame — a non-final variable could change after capture, causing unpredictable behavior.

Example

```
int count = 10;  // effectively final (never reassigned)
Runnable r = () -> System.out.println(count); // OK

int mutable = 5;
mutable++;  // reassigned -> no longer effectively final
// Runnable r2 = () -> System.out.println(mutable); // COMPILE ERROR

// Workaround: use array or AtomicInteger
int[] counter = {0};
Runnable r3 = () -> counter[0]++;  // OK: array ref is final
```

```
AtomicInteger atomicCount = new AtomicInteger(0);
Runnable r4 = () -> atomicCount.incrementAndGet(); // OK

// Instance fields are always accessible (not subject to this rule)
class Counter {
    int count = 0;
    Runnable increment = () -> count++; // OK: this is captured
}
```

Q27. What is the difference between map() and flatMap() in Streams? [Lambda] [Functional]

Answer

map() transforms each element to another element (1-to-1 mapping). flatMap() transforms each element to a Stream and then flattens all the streams into one (1-to-many). Use flatMap when each element maps to multiple elements or when dealing with nested structures like Optional, List, or Stream.

Example

```
List<String> words = List.of("Hello", "World");

// map: each String -> its length (1-to-1)
words.stream()
     .map(String::length)
     .forEach(System.out::println); // 5, 5

// flatMap: each String -> Stream of chars (1-to-many)
words.stream()
     .flatMap(s -> Arrays.stream(s.split("")))
     .forEach(System.out::print); // HelloWorld

// Nested lists: flatMap to flatten
List<List<Integer>> nested = List.of(
    List.of(1,2,3), List.of(4,5), List.of(6));

nested.stream()
      .flatMap(Collection::stream)
      .collect(Collectors.toList()); // [1,2,3,4,5,6]

// Optional.flatMap: chain optional-returning functions
Optional<String> opt = Optional.of("hello");
Optional<String> upper = opt.flatMap(s ->
    s.isEmpty() ? Optional.empty() : Optional.of(s.toUpperCase()));
System.out.println(upper); // Optional[HELLO]
```

Q28. Why is modifying a key object in a HashMap dangerous? [Collections] [Tricky]

Answer

HashMap stores entries based on hashCode() computed at insertion time. If you mutate an object that is used as a key such that its hashCode changes, the entry will be stored in the wrong bucket. Future lookups using the same (now mutated) object will compute the new hashCode and search the wrong bucket, failing to find the entry even though it exists.

Example

```
class MutableKey {
    int id;
    MutableKey(int id) { this.id = id; }
    @Override public int hashCode() { return id; }
    @Override public boolean equals(Object o) {
        return o instanceof MutableKey && id == ((MutableKey)o).id;
```

```
    }
}

Map<MutableKey, String> map = new HashMap<>();
MutableKey key = new MutableKey(1);
map.put(key, "original");
System.out.println(map.get(key)); // "original"

key.id = 2;  // MUTATE the key! hashCode changes!
System.out.println(map.get(key)); // null! Wrong bucket!

// Entry still exists in map but is now "lost"
map.entrySet().forEach(e ->
    System.out.println(e.getKey().id + "=" + e.getValue()));
// Still prints: 2=original (it's there but unfindable)

// Rule: use only immutable objects as HashMap keys
```

Q29. What is the difference between intermediate and terminal stream operations? [Streams] [Tricky]

Answer

Intermediate operations (filter, map, sorted, distinct, limit, skip) are lazy — they return a new Stream and do not execute until a terminal operation is called. Terminal operations (collect, forEach, count, findFirst, reduce, toArray) are eager — they trigger evaluation of the entire pipeline. This laziness enables short-circuiting and optimization.

Example

```
List<Integer> numbers = List.of(1,2,3,4,5,6,7,8,9,10);

// No output yet -- all intermediate (lazy)
Stream<Integer> pipeline = numbers.stream()
    .filter(n -> { System.out.println("filter: "+n); return n%2==0; })
    .map(n    -> { System.out.println("map: "+n); return n*n; });

// Terminal operation triggers execution
Optional<Integer> first = pipeline.findFirst();
// Prints: filter:1, filter:2, map:2 -- stops after first match!
// Only processes until first even number found

// Short-circuit: limit stops after n elements
numbers.stream()
    .map(n -> n * 2)
    .limit(3)          // stops after 3
    .collect(Collectors.toList()); // [2, 4, 6]

// Common mistake: reusing a stream
Stream<Integer> s = numbers.stream();
long count1 = s.count();
// long count2 = s.count(); // IllegalStateException: stream already consumed
```

Q30. What is the difference between reduce() and collect() in Streams? [Streams] [Tricky]

Answer

reduce() combines elements into a single immutable result using a binary operator — best for primitive types and simple reductions. collect() is a mutable reduction — it accumulates elements into a mutable container (like a List or Map). collect() with Collectors is more efficient for building collections because it avoids creating many intermediate objects.

Example

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);

// reduce: immutable accumulation
int sum = nums.stream().reduce(0, Integer::sum);
System.out.println(sum); // 15

Optional<Integer> product = nums.stream()
    .reduce((a, b) -> a * b);
System.out.println(product.get()); // 120

// collect: mutable container
List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList()); // [2, 4]

Map<Boolean, List<Integer>> partitioned = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
// {false=[1,3,5], true=[2,4]}

// collect with joining
String joined = Stream.of("a","b","c")
    .collect(Collectors.joining(", ", "[", "]"));
System.out.println(joined); // [a, b, c]
```

SECTION 4 — Exception Handling & Tricky Control Flow

Q31. What is the output when both catch and finally throw exceptions? [Exception] [Tricky]

Answer

If finally throws an exception, it completely replaces any exception thrown in try or catch — the original exception is lost silently. This is why you should never throw from finally. The finally block always executes (except System.exit() or JVM crash), and any exception in it suppresses the original.

Example

```
static void test() throws Exception {
    try {
        throw new RuntimeException("from try");
    } catch (RuntimeException e) {
        System.out.println("catch: " + e.getMessage());
        throw new RuntimeException("from catch");
    } finally {
        System.out.println("finally");
        throw new RuntimeException("from finally"); // DANGER!
    }
}

try {
    test();
} catch (Exception e) {
    System.out.println("outer: " + e.getMessage());
}
// Prints:
// catch: from try
// finally
// outer: from finally   <-- "from catch" is LOST!
```

```
// Java 7+ try-with-resources handles this with suppressed exceptions
```

Q32. What does return in a finally block do? [Exception] [Tricky]

Answer

A return in finally overrides any return in the try or catch block. The value from finally replaces the value that would have been returned. This is a subtle and confusing behavior. It also suppresses any exception that would have propagated. Avoid return, throw, break, or continue in finally blocks.

Example

```
static int test() {
    try {
        return 1;           // This return is overridden
    } finally {
        return 2;           // This return wins!
    }
}
System.out.println(test()); // 2, not 1!

// Exception suppressed by finally return:
static int dangerous() {
    try {
        throw new RuntimeException("error");
    } finally {
        return 42; // exception SILENTLY SUPPRESSED!
    }
}
System.out.println(dangerous()); // 42, no exception thrown!

// Rule: NEVER use return in finally blocks
// IntelliJ/SonarQube flag this as a code smell
```

Q33. What is the difference between checked and unchecked exceptions? [Exception] [Core]

Answer

Checked exceptions (extend Exception but not RuntimeException) must be handled or declared in the method signature. They represent recoverable conditions the caller should know about (IOException, SQLException). Unchecked exceptions (extend RuntimeException or Error) do not need to be declared; they represent programming errors or unrecoverable conditions (NullPointerException, ArrayIndexOutOfBoundsException, OutOfMemoryError).

Example

```
// Checked: must handle or declare
void readFile(String path) throws IOException {
    FileReader fr = new FileReader(path); // throws IOException
}

// Or handle it:
void readFileSafe(String path) {
    try {
        new FileReader(path);
    } catch (IOException e) {
        System.out.println("File not found: " + e.getMessage());
    }
}

// Unchecked: no declaration needed
void unsafe(String s) {
    s.length();  // throws NPE if s is null -- no declaration needed
```

```
}

// Error: unrecoverable (don't catch)
// StackOverflowError, OutOfMemoryError

// Modern practice: prefer unchecked exceptions
// Wrap checked in RuntimeException for cleaner APIs
throw new UncheckedIOException(ioException);
```

Q34. What is exception chaining and what is getCause()? [Exception] [Tricky]

▶ Answer

Exception chaining allows preserving the original cause when catching one exception and throwing another. Pass the original exception as the cause parameter to the new exception's constructor. getCause() retrieves the original exception. This preserves the full stack trace chain. Without chaining, the root cause is lost — making debugging difficult.

⌨ Example

```
class ServiceException extends RuntimeException {
    ServiceException(String msg, Throwable cause) {
        super(msg, cause);
    }
}

void connectDB() throws ServiceException {
    try {
        throw new SQLException("Connection refused");
    } catch (SQLException e) {
        // Chain: wrap original in higher-level exception
        throw new ServiceException("DB unavailable", e);
    }
}

try {
    connectDB();
} catch (ServiceException e) {
    System.out.println(e.getMessage());        // DB unavailable
    System.out.println(e.getCause());          // java.sql.SQLException: ...
    e.printStackTrace();                        // Full chain
}

// initCause() for post-construction chaining
RuntimeException re = new RuntimeException("outer");
re.initCause(new IOException("inner"));
```

Q35. How does try-with-resources work and what is the Suppressed exception mechanism? [Exception] [Java7]

▶ Answer

try-with-resources (Java 7+) automatically calls close() on AutoCloseable resources when the try block exits. If both the body and close() throw exceptions, the exception from close() is added as a suppressed exception to the primary exception (not lost like with finally). Retrieve suppressed exceptions with getSuppressed().

⌨ Example

```
// Resource auto-closed in all exit paths
try (BufferedReader br = new BufferedReader(new FileReader("f.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    System.out.println(e);
```

```
}
// br.close() called automatically

// Multiple resources: closed in reverse order
try (Connection conn = getConn(); Statement stmt = conn.createStatement()) {
    stmt.execute("SELECT 1");
}  // stmt.close() then conn.close()

// Suppressed exceptions
class CloseThrows implements AutoCloseable {
    public void close() throws Exception {
        throw new Exception("from close");
    }
}
try (CloseThrows c = new CloseThrows()) {
    throw new RuntimeException("from body");
} catch (Exception e) {
    System.out.println(e.getMessage());            // from body (primary)
    System.out.println(e.getSuppressed()[0]);      // from close (suppressed)
}
```

SECTION 5 — Concurrency Traps

Q36. What is the visibility problem in Java multithreading? [Concurrency] [Tricky]

▶ Answer

Without synchronization, changes made by one thread may not be visible to another thread due to CPU caching, instruction reordering, and compiler optimizations. Each thread may work with a cached copy of a variable. The volatile keyword guarantees visibility (changes are written directly to main memory and read from there) but does not guarantee atomicity.

⌨ Example

```
class VisibilityBug {
    boolean stop = false; // not volatile!

    void worker() {
        while (!stop) { }  // may loop forever
        System.out.println("Stopped");
    }

    void stopWorker() { stop = true; } // may not be seen!
}

// FIX: use volatile
volatile boolean stop = false;

// FIX: use synchronized
synchronized void stopWorker() { stop = true; }
synchronized boolean shouldStop() { return stop; }

// FIX: use AtomicBoolean
AtomicBoolean atomicStop = new AtomicBoolean(false);
void stopWorker2() { atomicStop.set(true); }
void worker2() { while (!atomicStop.get()) {} }
```

Q37. Why is volatile insufficient for i++ in a multithreaded context? [Concurrency] [Tricky]

Answer

volatile guarantees visibility but NOT atomicity. i++ is three operations: read i, increment it, write it back. Two threads can interleave these steps: both read the same value, both increment, both write — losing one increment. Use AtomicInteger.incrementAndGet() or synchronized for compound operations.

Example

```
class Counter {
    volatile int count = 0;
    void increment() { count++; } // NOT atomic! Race condition!
}

// Why volatile fails for count++:
// Thread A: read count (0)
// Thread B: read count (0)
// Thread A: increment to 1, write
// Thread B: increment to 1, write  <- lost update!

// FIX 1: synchronized method
class SafeCounter {
    int count = 0;
    synchronized void increment() { count++; }
    synchronized int get() { return count; }
}

// FIX 2: AtomicInteger (lock-free, preferred)
AtomicInteger atomicCount = new AtomicInteger(0);
atomicCount.incrementAndGet(); // atomic: read-modify-write

// FIX 3: LongAdder (highest throughput for counters)
LongAdder adder = new LongAdder();
adder.increment();
System.out.println(adder.sum());
```

Q38. What is the problem with double-checked locking in Java before Java 5? [Concurrency] [Tricky]

Answer

Before Java 5, the Java Memory Model did not guarantee that a non-null reference to an object meant the object's fields were fully initialized — partial construction was visible to other threads. In Java 5+, declaring the instance field as volatile fixes this because volatile establishes a happens-before relationship, ensuring full initialization is visible before the reference is published.

Example

```
// BROKEN pre-Java 5:
class BrokenSingleton {
    private static BrokenSingleton instance;  // not volatile!
    private int value;

    private BrokenSingleton() { this.value = 42; }

    public static BrokenSingleton get() {
        if (instance == null) {               // T1 and T2 both null
            synchronized (BrokenSingleton.class) {
                if (instance == null) {
                    instance = new BrokenSingleton();
                    // JVM may reorder: assign ref BEFORE constructor!
                }
            }
```

```
        }
        return instance; // T2 may get half-initialized object!
    }
}

// CORRECT Java 5+: volatile + double-check
private static volatile BrokenSingleton instance;

// BEST: initialization-on-demand holder
class SafeSingleton {
    private static class Holder {
        static final SafeSingleton INSTANCE = new SafeSingleton();
    }
    public static SafeSingleton get() { return Holder.INSTANCE; }
}
```

Q39. What is the difference between synchronized method and synchronized block? [Concurrency] [Core]

▶ Answer

A synchronized method acquires the lock on the entire object (or the class for static methods). A synchronized block allows specifying a different lock object, enabling finer-grained locking — for example, two independent operations in the same class can use different locks, allowing them to proceed concurrently.

⌨ Example

```
class Example {
    private final Object lockA = new Object();
    private final Object lockB = new Object();

    // Synchronized method: lock is "this"
    synchronized void methodA() {
        // Only one thread at a time for ANY synchronized method
    }

    // Synchronized block on different lock objects:
    // methodA and methodB can run concurrently!
    void methodB_Part1() {
        synchronized (lockA) { updateResourceA(); }
    }

    void methodB_Part2() {
        synchronized (lockB) { updateResourceB(); }
    }

    // Static synchronized: lock is Example.class
    static synchronized void staticMethod() { }

    // Equivalent to:
    static void staticMethodBlock() {
        synchronized (Example.class) { }
    }
}
```

Q40. What is ThreadLocal and when can it cause memory leaks? [Concurrency] [Tricky]

▶ Answer

ThreadLocal provides thread-scoped variables — each thread has its own independent copy. Commonly used for database connections, user sessions, and formatters. Memory leak risk: in thread pools, threads are reused

— ThreadLocal values survive between tasks. If the ThreadLocal holds a reference to a large object (or ClassLoader), it is never GC'd. Always call remove() after use.

Example

```
// Correct usage
ThreadLocal<SimpleDateFormat> sdf = ThreadLocal.withInitial(
    () -> new SimpleDateFormat("yyyy-MM-dd"));

String format(Date d) {
    return sdf.get().format(d); // thread-safe without sync
}

// MEMORY LEAK in thread pools:
ExecutorService pool = Executors.newFixedThreadPool(10);
pool.submit(() -> {
    threadLocal.set(new HeavyObject());
    doWork();
    // Thread returns to pool WITH HeavyObject still set!
    // HeavyObject never GC'd as long as thread lives
});

// FIX: always remove
pool.submit(() -> {
    try {
        threadLocal.set(new HeavyObject());
        doWork();
    } finally {
        threadLocal.remove(); // cleanup ALWAYS
    }
});
```

SECTION 6 — JVM Internals & Memory

Q41. What are the JVM memory areas and what goes where? [JVM] [Memory]

Answer

Heap: objects and class instances; shared by all threads; GC managed. Stack: each thread has its own stack of frames — local variables, operand stack, method references. Method Area / Metaspace (Java 8+): class metadata, static fields, method bytecode. PC Register: current instruction pointer per thread. Native Stack: JNI calls. PermGen was removed in Java 8 and replaced with Metaspace in native memory.

Example

```
// Heap: new objects go here
String s = new String("hello"); // String object on heap

// Stack: local variables, method frames
void method() {
    int local = 5;         // on stack frame
    String ref = s;        // reference on stack, object on heap
}

// Metaspace: class metadata
// Loaded class bytecode, static variables, method tables

// String Pool: part of heap (Java 7+)
// Was in PermGen before Java 7
String pooled = "hello"; // goes to string pool
```

```
// Memory settings
// -Xms<size>: initial heap size
// -Xmx<size>: max heap size
// -XX:MetaspaceSize=<size>: initial metaspace
// -Xss<size>: thread stack size

// Common: -Xms512m -Xmx2g -XX:MaxMetaspaceSize=512m
```

Q42. Explain the generations in the JVM Garbage Collector. [JVM] [GC]

Answer

The JVM heap is divided into generations based on the generational hypothesis: most objects die young. Young Generation: where new objects are created (Eden + two Survivor spaces); collected by Minor GC (fast, frequent). Old Generation (Tenured): objects that survived multiple GC cycles; collected by Major/Full GC (slow, infrequent). G1GC divides heap into equal-sized regions, not strict generational areas.

Example

```
// Young Generation
// Eden space: new objects allocated here
// S0, S1: survivor spaces

// Object lifecycle:
// 1. Allocated in Eden
// 2. Minor GC: live objects moved to S0
// 3. Next Minor GC: S0 live -> S1 (age++)
// 4. After threshold (default 15): promoted to Old Gen

// Types of GC:
// Minor GC: Young Gen only, Stop-The-World (STW), fast ~ms
// Major GC: Old Gen, slower
// Full GC:  Entire heap, longest pause

// Modern GC algorithms:
// Serial GC:     single thread, small apps
// Parallel GC:   multiple threads, throughput focus
// G1GC (Java 9 default): low latency, large heaps
// ZGC (Java 15+): sub-millisecond pauses
// Shenandoah:    concurrent compaction

// -XX:+UseG1GC  -XX:+UseZGC  -XX:+UseShenandoahGC
```

Q43. What is the difference between strong, soft, weak, and phantom references? [JVM] [Tricky]

Answer

Strong: regular references — object is GC-eligible only when no strong references exist. Soft: GC'd only when JVM needs memory — good for caches. Weak: GC'd at next GC cycle regardless of memory — used in WeakHashMap. Phantom: no access to object — used for cleanup actions after finalization. All except strong are in java.lang.ref.

Example

```
import java.lang.ref.*;

// Strong: not collected while referenced
Object strong = new Object();

// Soft: collected under memory pressure
SoftReference<byte[]> soft = new SoftReference<>(new byte[1024*1024]);
```

```
byte[] data = soft.get(); // null if collected

// Weak: collected at next GC
WeakReference<Object> weak = new WeakReference<>(new Object());
System.gc(); // hint to GC
System.out.println(weak.get()); // likely null after GC

// WeakHashMap: keys are weak references
Map<Object,String> map = new WeakHashMap<>();
Object key = new Object();
map.put(key, "value");
key = null;             // remove strong reference
System.gc();
System.out.println(map.size()); // 0 -- key GC'd

// Phantom: post-finalization cleanup (no get())
ReferenceQueue<Object> queue = new ReferenceQueue<>();
PhantomReference<Object> phantom = new PhantomReference<>(new Object(), queue);
```

Q44. What is the difference between == and equals() for primitive wrappers, and why does it matter for performance? [JVM] [Tricky]

Answer

The Integer cache (-128 to 127) means valueOf() reuses objects. New Integer objects created outside this range use heap memory. Using == on wrappers compares references, not values. Using .equals() always compares values. Autoboxing in loops can cause performance issues from repeated allocation — prefer primitives in performance-sensitive code.

Example

```
// Cache range: -128 to 127
Integer a = 50, b = 50;
System.out.println(a == b);       // true (cached)

Integer c = 200, d = 200;
System.out.println(c == d);       // false (not cached)
System.out.println(c.equals(d)); // true

// Performance: autoboxing in loop is expensive
Long sum = 0L;                    // BOXED Long
for (long i = 0; i < 1_000_000; i++) {
    sum += i;  // unbox sum, add, box result -- 1M allocations!
}

// Fix: use primitive
long sum2 = 0L;                   // primitive long
for (long i = 0; i < 1_000_000; i++) {
    sum2 += i;  // no boxing
}

// Check: Integer.valueOf vs new Integer (deprecated Java 9+)
// Integer.valueOf(5) returns cached; new Integer(5) always new obj
```

Q45. What is the class loading mechanism in Java? [JVM] [ClassLoader]

Answer

Java uses a delegation model: when a class is requested, the class loader first delegates to its parent before attempting to load it itself. Hierarchy: Bootstrap (loads rt.jar, native) -> Extension/Platform (ext libraries) ->

Application (classpath). This ensures core classes are loaded by trusted loaders. Custom class loaders can override this for plugins, hot-reloading, and isolation.

Example

```
// Class loaders hierarchy
ClassLoader appCL = Example.class.getClassLoader();
System.out.println(appCL);          // AppClassLoader
System.out.println(appCL.getParent()); // PlatformClassLoader (Java 11+)
System.out.println(appCL.getParent().getParent()); // null (Bootstrap)

// Custom class loader (for hot-reload / plugins)
class PluginLoader extends URLClassLoader {
    PluginLoader(URL[] urls) { super(urls, null); } // null parent!
    // null parent bypasses delegation -- loads fresh copy
}

// Class.forName: loads by name
Class<?> cls = Class.forName("com.example.MyClass");

// Class loading triggers:
// 1. new MyClass()
// 2. MyClass.staticField
// 3. MyClass.staticMethod()
// 4. Class.forName("MyClass")
// 5. Subclass loaded triggers superclass loading

// Static initializer runs once per ClassLoader at load time
class Init {
    static { System.out.println("Class loaded!"); }
}
```

SECTION 7 — String, StringBuilder & Interning

Q46. When should you use StringBuilder vs StringBuffer vs String concatenation? [String] [Performance]

Answer

String: immutable — each concatenation creates a new object (except compile-time constants). Use for 0-1 concatenations or when immutability is desired. StringBuilder: mutable, NOT thread-safe — use in single-threaded code for building strings in loops. StringBuffer: mutable, thread-safe (synchronized) — only when multiple threads share the builder (rare). Java compiler optimizes simple + concatenation with StringBuilder automatically.

Example

```
// String concatenation in loop: O(n^2) -- creates many objects
String result = "";
for (int i = 0; i < 10000; i++) {
    result += i;  // creates new String each time!
}

// StringBuilder: O(n) -- reuses buffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);  // amortized O(1) append
}
String r = sb.toString();
```

```
// Java 9+: String.format and + use StringConcatFactory
// "Hello " + name + "!" -> optimized by JIT/invokedynamic

// Compile-time concatenation of constants:
static final String PREFIX = "Hello, ";
static final String FULL = PREFIX + "World!"; // computed at compile time!

// Thread-safe: StringBuffer (rarely needed)
StringBuffer tsb = new StringBuffer();
tsb.append("thread-safe"); // synchronized append
```

Q47. What is String interning and when should you use intern()? [String] [Tricky]

Answer

String.intern() returns the canonical string from the pool. If the pool already contains the string, returns the existing instance; otherwise adds it and returns it. Interning lets you use == for comparison. In Java 7+, the string pool is on the heap (not PermGen), so it can hold more strings. Interning is useful when you have many duplicate strings to save memory.

Example

```
String a = new String("hello"); // NOT interned
String b = new String("hello"); // NOT interned
System.out.println(a == b);      // false

String ai = a.intern();          // intern a
String bi = b.intern();          // bi gets SAME interned obj
System.out.println(ai == bi);    // true

// Literal is automatically interned
String lit = "hello";
System.out.println(ai == lit);   // true (same pool object)

// Memory optimization: many duplicate strings
String[] rawData = fetchMillionRecords(); // many "US","EU","APAC"
for (int i = 0; i < rawData.length; i++) {
    rawData[i] = rawData[i].intern(); // deduplicate in pool
}
// Now == works and duplicates share memory

// Caution: intern() is not free -- hash lookup on pool
```

Q48. What changed in String implementation in Java 9 (Compact Strings)? [String] [Java9Plus]

Answer

Before Java 9, String stored characters as char[] (UTF-16, 2 bytes per char). Java 9 introduced Compact Strings: if all characters fit in Latin-1 (1 byte each), the string is stored as byte[] with LATIN1 encoding — halving memory for ASCII strings. If not, it uses UTF-16 byte[]. This is transparent to the API. Typical Java applications use 20-50% less heap.

Example

```
// Java 9+ internal representation (transparent to users)
String ascii = "hello";  // stored as byte[5] LATIN1 (1 byte/char)
String mixed = "héllo";  // stored as byte[10] UTF16  (2 bytes/char)

// Check coder (internal, reflective)
// LATIN1 = 0, UTF16 = 1
Field coder = String.class.getDeclaredField("coder");
coder.setAccessible(true);
```

```
System.out.println(coder.getByte(ascii)); // 0 (LATIN1)
System.out.println(coder.getByte(mixed)); // 1 (UTF16)

// Impact: ~50% memory reduction for ASCII strings
// No API changes: String.length(), charAt() work same

// Java 11: String.isBlank(), strip() (Unicode-aware)
System.out.println("   ".isBlank());     // true
System.out.println("  hi  ".strip());    // "hi" (Unicode)
System.out.println("  hi  ".trim());     // "hi" (char <= 32)

// Java 15: Text blocks
String json = """
    {"key": "value"}
    """;
```

Q49. What is the output? String s = "java"; s = s + " rocks"; What happens to the original "java" string? [String] [Tricky]

Answer

The original "java" String object is unchanged (Strings are immutable). The + operator creates a new String "java rocks" and the reference s is updated to point to it. The old "java" string (if it was created as a literal) stays in the string pool and is eligible for GC (if no other references). If it was interned, it stays in the pool indefinitely.

Example

```
String s = "java";           // "java" in string pool
System.identityHashCode(s); // e.g., 1234

s = s + " rocks";           // new String "java rocks" created
System.identityHashCode(s); // e.g., 5678 -- different object!

// "java" string: still in pool, s no longer references it
// Pool strings are GC roots -- not collected while pool exists

// What really happens behind the scenes:
// s + " rocks"
// = new StringBuilder(s).append(" rocks").toString()
// = new String("java rocks")

// Prove immutability:
String original = "hello";
String modified = original.replace('h', 'H');
System.out.println(original); // "hello" -- unchanged
System.out.println(modified); // "Hello" -- new object
System.out.println(original == modified); // false
```

Q50. What is the output? "hello" == "hel" + "lo"; vs "hello" == "hel" + variable; [String] [Tricky]

Answer

"hel" + "lo" is a compile-time constant expression involving two String literals — the compiler evaluates it to "hello" and interns it, so == returns true. "hel" + variable is a runtime concatenation using StringBuilder — it creates a new heap String, so == returns false. This is a subtle interning rule.

Example

```
String s1 = "hello";

// Compile-time constant: compiler creates "hello" and interns it
System.out.println(s1 == "hel" + "lo"); // true
```

```
// Runtime concatenation: new String on heap
String lo = "lo";
System.out.println(s1 == "hel" + lo);   // false

// final variable is compile-time constant:
final String LO = "lo";
System.out.println(s1 == "hel" + LO);   // true!
// final String constants are inlined by compiler

// Non-final: not a compile-time constant
String lo2 = "lo";  // not final
System.out.println(s1 == "hel" + lo2);  // false

// Summary:
// literal + literal   -> interned -> == may be true
// literal + variable  -> new String -> == false
// literal + final var -> interned -> == may be true
```

SECTION 8 — Java 8–21 Modern Features

Q51. What is Optional and what are its anti-patterns? [Java8] [Optional]

Answer

Optional<T> is a container that may or may not contain a value, designed to replace null returns. It forces callers to explicitly handle the absent case. Anti-patterns: calling get() without isPresent() (defeats the purpose), using Optional as a method parameter, using Optional in serializable fields, calling Optional.of(null) (throws NPE — use Optional.ofNullable()), nesting Optional<Optional<T>>.

Example

```
// Good usage: return type
Optional<User> findUser(int id) {
    return Optional.ofNullable(userMap.get(id));
}

findUser(42)
    .map(User::getName)         // transform if present
    .filter(n -> !n.isEmpty())  // filter
    .orElse("Anonymous");       // default value

// Anti-patterns:
// 1. get() without isPresent()
Optional<String> opt = Optional.empty();
opt.get();  // NoSuchElementException!

// 2. Optional as parameter (use overloading instead)
// void process(Optional<String> name) {} // bad

// 3. Optional.of(null) throws NPE
// Optional.of(null); // NPE!
// Optional.ofNullable(null); // OK -> empty Optional

// Good alternatives to get():
opt.orElse("default");
opt.orElseGet(() -> computeDefault());
opt.orElseThrow(() -> new RuntimeException("not found"));
```

```
opt.ifPresent(v -> process(v));
```

Q52. What is the difference between default and static methods in interfaces? [Java8] [Interface]

Answer

Default methods have an implementation in the interface and are inherited by implementing classes (can be overridden). They were added to allow interface evolution without breaking existing implementations. Static methods in interfaces are NOT inherited and cannot be overridden — they belong to the interface itself and are called via InterfaceName.method().

Example

```
interface Vehicle {
    // Abstract: must implement
    String getType();

    // Default: inherited, can override
    default String describe() {
        return "Vehicle: " + getType();
    }

    // Static: not inherited, called via interface
    static Vehicle create(String type) {
        return () -> type;
    }
}

class Car implements Vehicle {
    @Override public String getType() { return "Car"; }
    // describe() is inherited
    // Vehicle.create() is NOT accessible as create()
}

Car c = new Car();
System.out.println(c.describe());     // "Vehicle: Car"
// c.create("Truck");               // COMPILE ERROR: static
Vehicle v = Vehicle.create("Truck"); // called on interface
System.out.println(v.getType());     // "Truck"
```

Q53. What are Java Records and how do they differ from regular classes? [Java14Plus] [Records]

Answer

Records (Java 16+) are immutable data carriers. The compiler automatically generates: a canonical constructor, final private fields, getters (named after fields), equals(), hashCode(), and toString(). Records cannot extend other classes (implicitly extend Record), can implement interfaces, can have custom constructors (compact or explicit), and can have static fields and methods.

Example

```
// record: compact declaration
record Point(int x, int y) {}

// Equivalent verbose class
final class PointVerbose {
    private final int x, y;
    PointVerbose(int x, int y) { this.x = x; this.y = y; }
    int x() { return x; }
    int y() { return y; }
    @Override public boolean equals(Object o) { ... }
    @Override public int hashCode() { ... }
    @Override public String toString() { ... }
```

```
}

Point p = new Point(3, 4);
System.out.println(p.x());          // 3
System.out.println(p);              // Point[x=3, y=4]

// Compact constructor for validation
record Range(int min, int max) {
    Range {  // compact constructor
        if (min > max) throw new IllegalArgumentException();
    }
}

// Records in pattern matching (Java 21)
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + ", " + y);
}
```

Q54. What are sealed classes and how do they work with switch? [Java14Plus] [Switch]

Answer

Sealed classes (Java 17+) restrict which classes can extend or implement them using the permits clause. This enables exhaustive pattern matching in switch expressions — the compiler knows all possible subtypes and can verify coverage without a default case. They represent closed type hierarchies.

Example

```
// Sealed hierarchy
sealed interface Shape permits Circle, Rectangle, Triangle {}

record Circle(double radius) implements Shape {}
record Rectangle(double w, double h) implements Shape {}
record Triangle(double a, double b, double c) implements Shape {}

// Exhaustive switch (no default needed!)
double area(Shape shape) {
    return switch (shape) {
        case Circle c    -> Math.PI * c.radius() * c.radius();
        case Rectangle r -> r.w() * r.h();
        case Triangle t  -> {
            double s = (t.a()+t.b()+t.c())/2;
            yield Math.sqrt(s*(s-t.a())*(s-t.b())*(s-t.c()));
        }
    }; // no default: compiler knows all cases covered
}

// Adding a new Shape subtype would cause compile error in switch
// -> forces you to handle the new case explicitly
```

Q55. What are Virtual Threads (Project Loom, Java 21) and how do they differ from platform threads? [Java21] [Concurrency]

Answer

Virtual threads are lightweight threads managed by the JVM, not the OS. Platform threads map 1:1 to OS threads (~1MB stack each, creation is slow). Virtual threads are cheap (~1KB, millions possible) and are multiplexed onto a small pool of carrier threads. When a virtual thread blocks on I/O, the carrier thread is freed to run another virtual thread. Ideal for high-concurrency I/O servers.

Example

```
// Platform thread: maps to OS thread, expensive
Thread platform = new Thread(() -> System.out.println("platform"));
platform.start();

// Virtual thread: lightweight, managed by JVM (Java 21)
Thread vt = Thread.ofVirtual().start(() -> System.out.println("virtual"));

// Create millions of virtual threads:
List<Thread> threads = new ArrayList<>();
for (int i = 0; i < 1_000_000; i++) {
    threads.add(Thread.ofVirtual().start(() -> {
        Thread.sleep(Duration.ofSeconds(1)); // blocks carrier, not OS thread
    }));
}
threads.forEach(t -> t.join());

// ExecutorService with virtual threads
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    exec.submit(() -> handleRequest(request));
}

// Key: virtual threads enable synchronous-style code
// with async scalability -- no callbacks or reactive needed
```

Q56. What is the Java Module System (JPMS) introduced in Java 9? [Java9Plus] [Modules]

▶ Answer

JPMS (Project Jigsaw) partitions Java code into modules — each with a module-info.java declaring what it exports and what it requires. Benefits: strong encapsulation (internal APIs not accessible), reliable configuration (no classpath hell), smaller JRE images (only include needed modules). The JDK itself is modularized (java.base, java.sql, etc.).

⌨ Example

```
// module-info.java
module com.myapp {
    requires java.sql;           // dependency
    requires transitive java.logging; // transitive: others get it too
    exports com.myapp.api;        // public API
    exports com.myapp.model to com.myapp.dao; // qualified export
    opens com.myapp.config to spring.core; // reflection access
}

// Without export, com.myapp.internal is not accessible
// even if class is public

// List JDK modules
// java --list-modules

// Minimal JRE image with jlink
// jlink --module-path $JAVA_HOME/jmods \
//   --add-modules java.base,java.sql,com.myapp \
//   --output custom-jre

// Automatic module: jar on module-path without module-info
// Named by jar filename with hyphens -> dots
// "log4j-2.jar" -> module name "log4j.2"
```

Q57. What are method references and what are the four types? [Java8] [Functional]

Answer

Method references are shorthand for lambdas that just call a method. Four types: (1) Static method: ClassName::staticMethod. (2) Instance method of a particular instance: instance::method. (3) Instance method of an arbitrary instance of a type: ClassName::instanceMethod (first parameter becomes the receiver). (4) Constructor: ClassName::new.

Example

```
// 1. Static method reference
Function<String, Integer> parse = Integer::parseInt;
System.out.println(parse.apply("42")); // 42

// 2. Instance method of a specific instance
String prefix = "Hello, ";
Function<String, String> greet = prefix::concat;
System.out.println(greet.apply("World")); // "Hello, World"

// 3. Instance method of arbitrary instance
// First parameter becomes "this"
Function<String, String> upper = String::toUpperCase;
System.out.println(upper.apply("hello")); // "HELLO"

Comparator<String> cmp = String::compareTo;

// 4. Constructor reference
Supplier<ArrayList<String>> listFactory = ArrayList::new;
ArrayList<String> list = listFactory.get();

Function<Integer, int[]> arrayFactory = int[]::new;
int[] arr = arrayFactory.apply(5); // new int[5]
```

Q58. What is the difference between sequential and parallel streams? [Java8] [Streams]

Answer

Sequential streams process elements one by one in order. Parallel streams use ForkJoinPool.commonPool() to split work across multiple CPU cores. Parallel can be faster for large datasets with expensive operations but adds overhead for small data. Be careful with stateful operations (sorted, distinct), non-thread-safe collectors, and operations that depend on encounter order.

Example

```
List<Integer> numbers = IntStream.rangeClosed(1, 10_000_000)
    .boxed().collect(Collectors.toList());

// Sequential: processes in order
long seqSum = numbers.stream()
    .mapToLong(Integer::longValue)
    .sum();

// Parallel: splits across ForkJoinPool.commonPool()
long parSum = numbers.parallelStream()
    .mapToLong(Integer::longValue)
    .sum();

// Pitfall: parallel with non-thread-safe structures
List<Integer> unsafeList = new ArrayList<>();
numbers.parallelStream().forEach(unsafeList::add); // DATA RACE!

// Correct: use thread-safe collector
List<Integer> safeList = numbers.parallelStream()
    .collect(Collectors.toList()); // thread-safe

// Control thread pool
ForkJoinPool custom = new ForkJoinPool(4);
```

```
custom.submit(() -> numbers.parallelStream().count()).get();
```

Q59. What is pattern matching for instanceof (Java 16) and switch (Java 21)? [Java16Plus] [Pattern]

Answer

Pattern matching instanceof eliminates explicit casting: if (obj instanceof String s) automatically casts obj to String as s within the true branch. Switch pattern matching (Java 21) enables type-based switch cases with patterns, guarded patterns (when clause), and null handling — replacing complex if-else chains.

Example

```
// OLD style: explicit cast
if (obj instanceof String) {
    String s = (String) obj;  // redundant cast
    System.out.println(s.length());
}

// Java 16 pattern matching instanceof
if (obj instanceof String s) {  // s is bound here
    System.out.println(s.length());
}

// Java 21 switch pattern matching
String describe(Object o) {
    return switch (o) {
        case Integer i when i < 0 -> "negative: " + i; // guarded
        case Integer i             -> "positive: " + i;
        case String s              -> "string: " + s;
        case null                  -> "null";
        default                    -> "other: " + o;
    };
}

System.out.println(describe(-5));    // negative: -5
System.out.println(describe("hi")); // string: hi
System.out.println(describe(null)); // null
```

Q60. What are text blocks and what are their indentation rules? [Java14Plus] [Text]

Answer

Text blocks (Java 15+) are multi-line string literals delimited by triple quotes. The compiler automatically strips common leading whitespace (based on the least-indented line). The closing """ position controls re-indentation. Trailing spaces are stripped. Use \s to preserve a trailing space, and \ for line continuation (no newline).

Example

```
// Text block: automatically handles indentation
String json = """
        {
            "name": "Alice",
            "age": 30
        }
        """;
// Strips 8 leading spaces (based on closing """)
// Result: {     "name": "Alice",     "age": 30 }

// No trailing newline: close """ on last content line
String noTrail = """
        Hello\
        World""";
```

```
// "HelloWorld"

// Preserve trailing space with \s
String html = """
        <p>   \s
        </p>""";

// formatted() method
String sql = """
        SELECT * FROM %s WHERE id = %d
        """.formatted("users", 42);
```

SECTION 9 — Design Patterns, Best Practices & Gotchas

Q61. What is the static initialization order problem? [Gotcha] [Static]

Answer

Static initializers run in textual order within a class, but the order between classes is determined by class loading. If class A's static field references class B, and B references A, there is a circular dependency — one of them will see the other partially initialized (zero/null values). This is the "static initialization order fiasco" — the Java equivalent of C++'s SIOF.

Example

```
class A {
    static final int VALUE = B.getB() + 1;
    static int getA() { return 10; }
}

class B {
    static final int VALUE = A.getA() + 1;  // A not yet init!
    static int getB() { return 20; }
}

// If A is loaded first:
// A.VALUE = B.getB() + 1 -> triggers B loading
// B.VALUE = A.getA() + 1  -> A.getA() returns 10 (method, not field)
// B.VALUE = 11
// A.VALUE = 20 + 1 = 21

// If B is loaded first, results differ!

// Safe: use lazy initialization with inner class
class SafeA {
    private static class Init {
        static final int VALUE = 42; // no circular dependency
    }
    static int value() { return Init.VALUE; }
}
```

Q62. What are defensive copies and when are they needed? [Gotcha] [Immutable]

Answer

A defensive copy is a new copy of a mutable object made to prevent external modification. Needed when: storing mutable parameters (constructor), returning mutable fields. Without defensive copies, an immutable-looking class can have its internal state mutated by callers who hold references to the mutable objects.

Example

```
import java.util.Date;

// WITHOUT defensive copy: mutable internals exposed
class Period {
    private final Date start, end;
    Period(Date s, Date e) { start = s; end = e; }
    Date getStart() { return start; }
}

Date d = new Date();
Period p = new Period(d, new Date());
d.setTime(0);  // MUTATES p.start -- broken "immutable"!

// WITH defensive copy: safe
class SafePeriod {
    private final Date start, end;
    SafePeriod(Date s, Date e) {
        this.start = new Date(s.getTime()); // copy
        this.end   = new Date(e.getTime()); // copy
    }
    Date getStart() { return new Date(start.getTime()); } // copy
}

// Modern: use Instant (immutable) instead of Date
record SafePeriod2(Instant start, Instant end) {}
```

Q63. Why should you prefer enum over int constants? [Gotcha] [Enum]

Answer

int constants (static final int) have no type safety — any int can be passed. Enums provide compile-time type safety, meaningful toString(), ability to add methods and fields, switch support, and namespace. Enums are classes — they can implement interfaces, have abstract methods, and carry per-constant behavior.

Example

```
// BAD: int constants
static final int MON=0, TUE=1, WED=2, THU=3, FRI=4, SAT=5, SUN=6;
void schedule(int day) { ... } // any int accepted -- unsafe!
schedule(42);  // compiles but meaningless

// GOOD: enum
enum Day {
    MON, TUE, WED, THU, FRI, SAT, SUN;
    boolean isWeekend() { return this==SAT || this==SUN; }
}

void schedule(Day day) { ... } // type-safe
// schedule(42);  // COMPILE ERROR

System.out.println(Day.MON);           // "MON" (toString)
System.out.println(Day.MON.name());    // "MON"
System.out.println(Day.MON.ordinal()); // 0
System.out.println(Day.MON.isWeekend()); // false

// EnumSet / EnumMap for efficient enum-keyed structures
Set<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
Map<Day, String> schedule = new EnumMap<>(Day.class);
```

Q64. What are the rules for Comparable and Comparator contracts? [Gotcha] [Comparable]

Answer

Comparable.compareTo() must be: antisymmetric (x.compareTo(y) < 0 implies y.compareTo(x) > 0), transitive, and consistent with equals() (recommended but not required). Violating these breaks sorted collections and algorithms. Comparator must be consistent too. Common mistake: using subtraction (a-b) for integer comparison — overflows for large values.

Example

```
// WRONG: subtraction overflows for large values!
Comparator<Integer> buggy = (a, b) -> a - b;
// Integer.MIN_VALUE - 1 overflows to positive!

// CORRECT: use Integer.compare()
Comparator<Integer> safe = Integer::compare;

// Comparable implementation
class Employee implements Comparable<Employee> {
    String name; int salary;

    @Override
    public int compareTo(Employee other) {
        // Primary: salary descending
        int cmp = Integer.compare(other.salary, this.salary);
        if (cmp != 0) return cmp;
        // Secondary: name ascending
        return this.name.compareTo(other.name);
    }
}

// Java 8 Comparator chaining
Comparator<Employee> comp = Comparator
    .comparingInt(Employee::getSalary).reversed()
    .thenComparing(Employee::getName);
```

Q65. What are serialVersionUID and the risks of Java serialization? [Gotcha] [Serialization]

Answer

serialVersionUID is a version identifier for Serializable classes. If it changes (or is not declared, causing auto-generation), deserialization of old data throws InvalidClassException. Security risks: deserialization of untrusted data can execute arbitrary code (gadget chains). Modern best practice: avoid Java serialization for external data; use JSON, Protocol Buffers, or records with Jackson.

Example

```
import java.io.*;

class User implements Serializable {
    private static final long serialVersionUID = 1L; // explicit
    private String name;
    private int age;

    // transient: not serialized
    transient String password; // excluded from serialization
}

// Serialize
try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream("user.ser"))) {
    oos.writeObject(new User("Alice", 30));
```

```
}

// Deserialize
try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream("user.ser"))) {
    User u = (User) ois.readObject();
}

// RISK: deserialization of untrusted data
// Use ObjectInputFilter to whitelist classes (Java 9+)
ObjectInputFilter.Config.setSerialFilter(
    ObjectInputFilter.allowFilter(c -> c == User.class, Status.REJECTED));
```

Q66. What is the Builder pattern and when should you use it? [Design] [Core]

▶ Answer

The Builder pattern constructs complex objects step-by-step. Use it when: a constructor has many parameters (telescoping constructor problem), some parameters are optional, the construction process must allow different configurations, or you want immutable objects with many fields. It improves readability and prevents invalid states.

⌨ Example

```
class HttpRequest {
    private final String url;
    private final String method;
    private final Map<String,String> headers;
    private final String body;
    private final int timeout;

    private HttpRequest(Builder b) {
        this.url     = b.url;
        this.method  = b.method;
        this.headers = Collections.unmodifiableMap(b.headers);
        this.body    = b.body;
        this.timeout = b.timeout;
    }

    static class Builder {
        private final String url;
        private String method = "GET";
        private Map<String,String> headers = new HashMap<>();
        private String body;
        private int timeout = 30_000;

        Builder(String url) { this.url = url; }
        Builder method(String m) { this.method = m; return this; }
        Builder header(String k, String v) { headers.put(k,v); return this; }
        Builder body(String b) { this.body = b; return this; }
        Builder timeout(int ms) { this.timeout = ms; return this; }
        HttpRequest build() { return new HttpRequest(this); }
    }
}

HttpRequest req = new HttpRequest.Builder("https://api.example.com")
    .method("POST").header("Content-Type","application/json")
    .body("{"key":"value"}").timeout(5000).build();
```

Q67. What are the hidden dangers of Java arrays with generics? [Gotcha] [Array]

Answer

Java arrays are covariant (String[] is a subtype of Object[]), while generics are invariant (List<String> is NOT a subtype of List<Object>). This means arrays can fail at runtime with ArrayStoreException while generics fail at compile time. You should prefer collections over arrays for type-safe generic code.

Example

```
// Arrays: covariant -- compiles but may throw at runtime
String[] strings = new String[3];
Object[] objects = strings;  // OK: covariant
objects[0] = "hello";        // OK
objects[1] = 42;             // ArrayStoreException at RUNTIME!

// Generics: invariant -- fail at compile time (safer)
List<String> strList = new ArrayList<>();
// List<Object> objList = strList; // COMPILE ERROR

// Generic arrays: not allowed
// List<String>[] arr = new List<String>[10]; // COMPILE ERROR

// Workaround: raw type or wildcard (unsafe)
@SuppressWarnings("unchecked")
List<String>[] arr = (List<String>[]) new List[10]; // unchecked cast

// Prefer: List<List<String>> for nested collections
List<List<String>> nestedList = new ArrayList<>();
```

Q68. What are the pitfalls of varargs with generics? [Gotcha] [Varargs]

Answer

Varargs with generic types creates a generic array internally (heap pollution). The compiler warns with "unchecked" or "potential heap pollution". If the varargs array is passed to or from a type-unsafe context, ClassCastException can occur at unexpected points. Java 7+ @SafeVarargs suppresses this warning when the method does not do anything unsafe with the array.

Example

```
// Unsafe: generic varargs can cause heap pollution
static <T> List<T> listOf(T... items) {
    return Arrays.asList(items); // items is T[], but stored as Object[]
}

// Heap pollution example:
static void dangerous(List<String>... stringLists) {
    Object[] array = stringLists;  // generic array -> Object[]
    array[0] = List.of(42);        // insert Integer List
    String s = stringLists[0].get(0); // ClassCastException here!
}

// Safe varargs: method only reads, doesn't expose array
@SafeVarargs
static <T> List<T> safeCombine(List<T>... lists) {
    List<T> result = new ArrayList<>();
    for (List<T> list : lists) result.addAll(list);
    return result;
}

// Best: use Collection parameter instead of varargs
static <T> List<T> combine(Collection<List<T>> lists) { ... }
```

Q69. What are the problems with the Cloneable interface? [Gotcha] [CloneEquals]

Answer

Cloneable is a broken API: it does not declare clone(), so you cannot call it without casting. Object.clone() is protected and creates a shallow copy — mutable fields are shared between original and clone. The clone() method does not call constructors, bypassing invariant checking. Use copy constructors or static factory methods instead.

Example

```
// The Cloneable/clone API is broken:
class Stack<E> implements Cloneable {
    Object[] elements;
    int size;

    @Override
    protected Stack<E> clone() throws CloneNotSupportedException {
        Stack<E> result = (Stack<E>) super.clone(); // shallow copy
        result.elements = elements.clone(); // must deep copy array
        return result;
    }
}

// PROBLEMS:
// 1. throws checked exception -- bad for use in lambdas
// 2. shallow by default -- bugs with mutable fields
// 3. no constructor called -- invariants not checked

// BETTER: copy constructor
class SafeStack<E> {
    Object[] elements;
    SafeStack(SafeStack<E> original) {
        this.elements = original.elements.clone();
    }
}

// BETTER: static factory
static SafeStack<E> copyOf(SafeStack<E> original) {
    return new SafeStack<>(original);
}
```

Q70. What can reflection do and what are its dangers? [Gotcha] [Reflection]

Answer

Reflection allows: inspecting classes, methods, fields at runtime; creating instances without calling constructors; accessing private members; invoking methods by name. Dangers: bypasses access control (security risk), breaks encapsulation, prevents compiler optimizations, is slow (JIT cannot inline), and breaks with Java modules (JPMS). Avoid reflection in application code; use it only for frameworks.

Example

```
import java.lang.reflect.*;

// Access private field
class Secret { private String secret = "TOP SECRET"; }

Secret obj = new Secret();
Field f = Secret.class.getDeclaredField("secret");
f.setAccessible(true);  // bypass private!
System.out.println(f.get(obj));  // "TOP SECRET"

// Invoke private method
Method m = Secret.class.getDeclaredMethod("privateMethod");
m.setAccessible(true);
```

```
m.invoke(obj);

// Create instance without constructor
// (using Unsafe -- very dangerous)
Unsafe unsafe = Unsafe.getUnsafe();
Secret s = (Secret) unsafe.allocateInstance(Secret.class);

// JPMS blocks: modules require opens declaration
// module-info: opens com.myapp to framework.core;

// Performance: ~50-100x slower than direct invocation
// Use MethodHandles for faster reflective access
MethodHandles.Lookup lookup = MethodHandles.lookup();
```

SECTION 10 — Expert Tricky Output & Edge Cases

Q71. What is the output? List<String> list = List.of("a"); list.add("b"); [Tricky] [Output]

Answer

It throws UnsupportedOperationException. List.of() (Java 9+) creates an unmodifiable list. Attempts to add, remove, or set elements throw UnsupportedOperationException. Same for Map.of(), Set.of(). Use new ArrayList<>(List.of(...)) to get a mutable copy.

Example

```
List<String> immutable = List.of("a", "b", "c");
try {
    immutable.add("d");  // UnsupportedOperationException!
} catch (UnsupportedOperationException e) {
    System.out.println("Cannot modify List.of()");
}

// Also immutable:
List<String> also = Arrays.asList("a","b");
also.set(0, "x"); // allowed: set is OK for asList
try {
    also.add("c"); // UnsupportedOperationException!
} catch (UnsupportedOperationException e) {}

// Mutable copy:
List<String> mutable = new ArrayList<>(List.of("a","b","c"));
mutable.add("d");  // OK

// Collections.unmodifiableList: wraps existing list
List<String> unmod = Collections.unmodifiableList(mutable);
// unmod.add("e"); // UnsupportedOperationException
// mutable.add("e"); // OK: view reflects change
```

Q72. What is NaN and how does it behave in Java comparisons? [Tricky] [NaN]

Answer

NaN (Not a Number) is the result of undefined floating-point operations (0.0/0.0, Math.sqrt(-1)). NaN is not equal to itself — all comparisons involving NaN return false (including NaN == NaN). Use Double.isNaN() to test for NaN. NaN in an array sorts to the end with Arrays.sort() (natural ordering handles it).

Example

```
double nan = Double.NaN;
double nan2 = 0.0 / 0.0;
double nan3 = Math.sqrt(-1);

System.out.println(nan == nan);   // false! NaN != NaN
System.out.println(nan != nan);   // true
System.out.println(nan < 0);      // false
System.out.println(nan > 0);      // false
System.out.println(nan == 0);     // false

// CORRECT: check with isNaN()
System.out.println(Double.isNaN(nan)); // true

// NaN in collections
List<Double> list = new ArrayList<>(List.of(3.0, nan, 1.0, nan, 2.0));
Collections.sort(list); // NaN goes to end
System.out.println(list); // [1.0, 2.0, 3.0, NaN, NaN]

// Double.compare handles NaN consistently
System.out.println(Double.compare(nan, 0.0)); // positive (NaN > all)
```

Q73. What is the output? for (int i = 0; i < 10; i++) { if (i == 5) continue; if (i == 8) break; System.out.print(i + " "); } [Tricky] [Output]

Answer

Output: 0 1 2 3 4 6 7. continue skips to the next iteration (skipping i==5). break exits the loop entirely (stopping at i==8, before printing 8). This tests understanding of loop control flow.

Example

```
for (int i = 0; i < 10; i++) {
    if (i == 5) continue; // skip 5, go to i=6
    if (i == 8) break;    // stop loop, 8 not printed
    System.out.print(i + " ");
}
// Output: 0 1 2 3 4 6 7

// Labeled break/continue
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer; // skip to next i
        if (i == 2) break outer;    // exit outer loop
        System.out.print(i + "" + j + " ");
    }
}
// Output: 00 10
// i=0,j=0: prints 00; j=1: continue outer (skip j=2)
// i=1,j=0: prints 10; j=1: continue outer
// i=2: break outer exits
```

Q74. What is the difference between static nested class and inner class? [Tricky] [Inner Class]

Answer

A static nested class does not hold a reference to its outer class — it can be instantiated without an outer instance. An inner class (non-static nested class) implicitly holds a reference to its outer class instance. This means inner classes can cause memory leaks: the outer class is kept alive as long as the inner class instance exists. Use static nested classes whenever the inner class does not need outer class access.

Example

```
class Outer {
    int x = 10;

    // INNER class: holds reference to Outer instance
    class Inner {
        void print() {
            System.out.println(x); // accesses Outer.x
            System.out.println(Outer.this.x); // explicit
        }
    }

    // STATIC nested: no reference to Outer
    static class StaticNested {
        void print() {
            // Cannot access x directly -- no outer instance
            System.out.println("static nested");
        }
    }
}

// Instantiation:
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner(); // requires outer instance

Outer.StaticNested nested = new Outer.StaticNested(); // no outer needed

// Memory leak: inner class holds Outer ref
// Common in Android: Activity held alive by inner Handler
```

Q75. What is the output of String.format with %n vs \n? [Tricky] [Output]

Answer

%n is the platform-specific line separator (\r\n on Windows, \n on Unix). \n is always Unix line feed. For cross-platform code, prefer %n in format strings. In println(), the correct separator is always used. System.lineSeparator() returns the platform-specific one.

Example

```
// \n: always Unix newline (LF, \u000A)
System.out.print("line1\nline2\n");

// %n: platform-specific separator
String s = String.format("line1%nline2%n");
// On Windows: "line1\r\nline2\r\n"
// On Unix:    "line1\nline2\n"

// System.lineSeparator()
System.out.println(System.lineSeparator().equals("\n")); // true on Unix

// println: always uses platform separator
System.out.println("hello"); // equivalent to print("hello" + lineSep)

// Print to string: use %n for cross-platform
String msg = String.format("Error: %s%n", "file not found");

// Files.write: use System.lineSeparator() for text files
Files.writeString(path, "line1" + System.lineSeparator() + "line2");
```

Q76. What is the output of Arrays.equals vs equals on arrays? [Tricky] [Arrays]

Answer

arr1.equals(arr2) (or arr1 == arr2) does reference comparison — same as Object.equals(). Only true if the same array object. Arrays.equals(arr1, arr2) compares element-by-element. Arrays.deepEquals works for nested arrays. Always use Arrays.equals() or Arrays.deepEquals() for array content comparison.

Example

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};

System.out.println(a == b);              // false (diff objects)
System.out.println(a.equals(b));          // false (Object.equals)
System.out.println(Arrays.equals(a, b));  // true (element-wise)

// Multi-dimensional
int[][] m1 = {{1,2},{3,4}};
int[][] m2 = {{1,2},{3,4}};

System.out.println(Arrays.equals(m1, m2));     // false! (shallow)
System.out.println(Arrays.deepEquals(m1, m2)); // true  (deep)

// toString
System.out.println(a);                   // [I@1b6d3586 (identity)
System.out.println(Arrays.toString(a));  // [1, 2, 3]
System.out.println(Arrays.deepToString(m1)); // [[1, 2], [3, 4]]

// sort + binarySearch
int[] arr = {5,3,1,4,2};
Arrays.sort(arr);
int idx = Arrays.binarySearch(arr, 3); // 2
```

Q77. What is the output? Integer.parseInt("10", 2); [Tricky] [Output]

Answer

It returns 2. parseInt(String, int radix) parses the string as a number in the given base. "10" in base 2 (binary) = 1*2^1 + 0*2^0 = 2. This is commonly used for binary/hex parsing. Base can be 2 (binary) through 36 (0-9 + a-z).

Example

```
System.out.println(Integer.parseInt("10", 2));  // 2  (binary: 10 = 2)
System.out.println(Integer.parseInt("10", 8));  // 8  (octal: 10 = 8)
System.out.println(Integer.parseInt("10", 16)); // 16 (hex: 10 = 16)
System.out.println(Integer.parseInt("FF", 16)); // 255
System.out.println(Integer.parseInt("1010", 2));// 10 (binary 1010)

// Convert int to String in base:
System.out.println(Integer.toBinaryString(42));  // "101010"
System.out.println(Integer.toHexString(255));    // "ff"
System.out.println(Integer.toOctalString(8));    // "10"
System.out.println(Integer.toString(42, 2));     // "101010"

// NumberFormatException for invalid digits in base
try {
    Integer.parseInt("9", 8); // 9 is not valid in octal
} catch (NumberFormatException e) { System.out.println("NFE"); }
```

Q78. What is the output? int x = Integer.MAX_VALUE; x++; [Tricky] [Overflow]

Answer

x becomes Integer.MIN_VALUE (-2147483648). Integer arithmetic in Java is modular — it wraps around silently on overflow with no exception. This is a common source of bugs. Use Math.addExact() to get ArithmeticException on overflow, or use long for larger ranges.

Example

```
int x = Integer.MAX_VALUE;  // 2147483647
System.out.println(x + 1); // -2147483648 (Integer.MIN_VALUE)

// No exception -- silent overflow!
System.out.println(Integer.MAX_VALUE + 1); // -2147483648

// Detect overflow: Math.addExact
try {
    int safe = Math.addExact(Integer.MAX_VALUE, 1);
} catch (ArithmeticException e) {
    System.out.println("Overflow detected!"); // throws
}

// Also available:
Math.subtractExact(a, b);
Math.multiplyExact(a, b);
Math.toIntExact(longValue); // throws if doesn't fit in int

// Long overflow wraps too:
long l = Long.MAX_VALUE;
System.out.println(l + 1);  // Long.MIN_VALUE

// Safe: use BigInteger for arbitrary precision
BigInteger big = BigInteger.valueOf(Long.MAX_VALUE).add(BigInteger.ONE);
```

Q79. What is the difference between Runnable, Callable, and Supplier? [Tricky] [Functional]

Answer

Runnable: no args, no return value, no checked exceptions. Callable<V>: no args, returns V, can throw checked Exception. Supplier<T>: no args, returns T, no checked exceptions (unlike Callable). Use Callable with ExecutorService for tasks that return values and may fail. Use Supplier for lazy evaluation and factory patterns.

Example

```
// Runnable: void, no exception
Runnable r = () -> System.out.println("running");
r.run();

// Callable<V>: returns V, throws Exception
Callable<Integer> c = () -> {
    if (Math.random() < 0.5) throw new IOException("failed");
    return 42;
};

// Submit to executor -- handles result and exception
ExecutorService exec = Executors.newSingleThreadExecutor();
Future<Integer> fut = exec.submit(c);
try {
    System.out.println(fut.get()); // blocks; rethrows as ExecutionException
} catch (ExecutionException e) {
    System.out.println("Task failed: " + e.getCause());
}

// Supplier<T>: lazy evaluation
Supplier<List<String>> lazyList = ArrayList::new;
List<String> list = lazyList.get(); // creates only when called
```

```
// Optional.orElseGet uses Supplier
Optional.empty().orElseGet(() -> "default");
```

Q80. What is the output? char c = 'A'; System.out.println(c + 1); [Tricky] [Output]

Answer

It prints 66. Adding an int to a char performs arithmetic — the char is implicitly widened to int (65 for 'A') and 65 + 1 = 66. The result is an int, not a char. To get 'B', you must cast: (char)(c + 1). This is a common confusing Java widening/promotion rule.

Example

```
char c = 'A';
System.out.println(c + 1);       // 66 (int arithmetic)
System.out.println((char)(c+1)); // B (cast back to char)

// Char arithmetic
System.out.println('A' + 'B');   // 131 (both widened to int)
System.out.println("" + 'A');    // "A" (String context, no widening)

// Char is 16-bit unsigned: 0 to 65535
char max = Character.MAX_VALUE;   // \uFFFF = 65535
System.out.println((int)max);     // 65535

// Arithmetic promotion rules:
// byte + byte  -> int
// short + short -> int
// char + int   -> int
// int + long   -> long
// long + float -> float
// float + double -> double

// For loop over characters
for (char ch = 'A'; ch <= 'Z'; ch++) {
    System.out.print(ch); // ABCDE...Z
}
```

Q81. What is the output? Collections.synchronizedList and iteration pitfall. [Tricky] [Concurrency]

Answer

Collections.synchronizedList wraps a list making each individual method call synchronized. However, iteration is NOT atomic — between hasNext() and next() calls, another thread can modify the list, causing ConcurrentModificationException. You must manually synchronize on the list during iteration.

Example

```
List<String> synced = Collections.synchronizedList(new ArrayList<>());
synced.add("a");
synced.add("b");
synced.add("c");

// UNSAFE: iteration not synchronized
// Another thread calling synced.remove() between iterations -> CME
for (String s : synced) { // potential CME in multi-threaded env
    System.out.println(s);
}

// CORRECT: synchronize on the list during iteration
synchronized (synced) {
    for (String s : synced) {
```

```
        System.out.println(s);
    }
}

// Better: use CopyOnWriteArrayList for read-heavy
List<String> cowList = new CopyOnWriteArrayList<>(List.of("a","b","c"));
for (String s : cowList) {  // iterates snapshot -- no CME
    cowList.add("d");       // allowed: modifies underlying, not snapshot
}
```

Q82. What is the output of a forEach with a conditional return? [Tricky] [Nested Lambda]

Answer

In a lambda inside forEach, return exits the lambda, not the enclosing method. This is different from a regular for loop where return exits the method. If you want to exit the method from inside a forEach lambda, you must throw an exception, use Stream.anyMatch, or use a traditional for loop.

Example

```
static void findFirst(List<Integer> list) {
    list.forEach(n -> {
        if (n == 3) return; // returns from LAMBDA, not method!
        System.out.print(n + " ");
    });
    System.out.println("done");
}

findFirst(List.of(1,2,3,4,5));
// Output: 1 2 4 5 done
// 3 is skipped (return from lambda)
// But method continues after forEach!

// To exit method early: use traditional for
static void findFirst2(List<Integer> list) {
    for (int n : list) {
        if (n == 3) return; // exits method
        System.out.print(n + " ");
    }
}
// Output: 1 2

// Or use Stream.findFirst()
list.stream().filter(n -> n < 3).findFirst().ifPresent(n ->
    System.out.println("Found: " + n));
```

Q83. What is the output? super() call rules in constructors. [Tricky] [Inheritance]

Answer

If a subclass constructor does not explicitly call super() or this(), the compiler inserts super() as the first statement. This calls the superclass no-arg constructor. If the superclass does not have a no-arg constructor, it is a compile error. The super() call must be the first statement in a constructor — nothing can precede it.

Example

```
class Animal {
    String sound;
    Animal() {
        sound = "...";
        System.out.println("Animal()");
    }
    Animal(String s) {
```

```
        sound = s;
        System.out.println("Animal(" + s + ")");
    }
}

class Dog extends Animal {
    Dog() {
        // Compiler inserts: super(); implicitly
        System.out.println("Dog()");
    }
    Dog(String s) {
        super(s);  // explicit -- must be FIRST statement
        System.out.println("Dog(" + s + ")");
    }
}

new Dog();
// Prints: Animal()   Dog()

new Dog("woof");
// Prints: Animal(woof)   Dog(woof)

// super() must be first: cannot do this:
// Dog() { int x = 5; super(); } // COMPILE ERROR
```

Q84. What is variable shadowing and what does "this" resolve? [Tricky] [Scope]

▶ Answer

Variable shadowing occurs when a local variable or parameter has the same name as a field. The local variable shadows the field. Use this.fieldName to explicitly access the field. Shadowing applies to: parameters in constructors/methods, local variables in blocks. It does not apply to superclass fields (use super.fieldName).

⌨ Example

```
class Counter {
    int count = 10;  // field

    void setCount(int count) {  // parameter shadows field
        count = count;          // BUG: assigns param to itself!
        // this.count still = 10
    }

    void setCountCorrect(int count) {
        this.count = count;     // CORRECT: field = parameter
    }

    void localShadow() {
        int count = 99;         // local shadows field
        System.out.println(count);      // 99 (local)
        System.out.println(this.count); // 10 (field)
    }
}

Counter c = new Counter();
c.setCount(50);            // BUG: count not updated
System.out.println(c.count); // 10 (unchanged!)

c.setCountCorrect(50);
System.out.println(c.count); // 50
```

Q85. What is the output? System.out.println(Math.min(Integer.MIN_VALUE, 0)); [Tricky] [Output]

Answer

It prints -2147483648 (Integer.MIN_VALUE). Math.min returns the smaller of the two values. Integer.MIN_VALUE (-2147483648) is less than 0. This is a straightforward question but often confuses candidates who forget that Integer.MIN_VALUE is negative.

Example

```
System.out.println(Integer.MIN_VALUE);   // -2147483648
System.out.println(Integer.MAX_VALUE);   //  2147483647

System.out.println(Math.min(Integer.MIN_VALUE, 0)); // -2147483648
System.out.println(Math.max(Integer.MAX_VALUE, 0)); //  2147483647

// Negating MIN_VALUE overflows!
System.out.println(-Integer.MIN_VALUE);   // -2147483648 (overflow!)
System.out.println(Math.abs(Integer.MIN_VALUE)); // -2147483648 (same!)

// Safe abs for all values:
long safeAbs = Math.abs((long) Integer.MIN_VALUE); // 2147483648L

// Distances between MIN/MAX:
long range = (long) Integer.MAX_VALUE - Integer.MIN_VALUE;
System.out.println(range); // 4294967295 (2^32 - 1)
```

Q86. What are the differences between Comparator.comparing and compareTo? [Tricky] [Java8]

Answer

compareTo() is an instance method on Comparable objects — called on one object with another as parameter. Comparator.comparing() is a static factory returning a Comparator based on a key extractor. The latter is more flexible: supports chaining (thenComparing), nulls first/last, and works without modifying the class. Use Comparator.comparing for complex sort logic.

Example

```
class Person {
    String name; int age;
    Person(String name, int age) {
        this.name = name; this.age = age;
    }
    String getName() { return name; }
    int getAge() { return age; }
}

List<Person> people = List.of(
    new Person("Bob",30), new Person("Alice",25), new Person("Bob",20));

// Comparator.comparing: flexible chaining
people.stream()
    .sorted(Comparator.comparing(Person::getName)
                      .thenComparingInt(Person::getAge))
    .forEach(p -> System.out.println(p.name + " " + p.age));
// Alice 25, Bob 20, Bob 30

// Nulls handling
Comparator<Person> nullSafe = Comparator.comparing(
    Person::getName, Comparator.nullsFirst(Comparator.naturalOrder()));

// Reversed
Comparator<Person> reversed = Comparator
    .comparingInt(Person::getAge).reversed();
```

Q87. Can you catch Error in Java? Should you? [Tricky] [Exception]

▶ Answer

Yes, you can catch Error and Throwable — Java does not prevent it syntactically. However, you almost never should: Errors indicate JVM-level problems (OutOfMemoryError, StackOverflowError) from which recovery is generally impossible. Catching Error may mask serious issues, leave the JVM in an inconsistent state, and prevent proper crash reporting. The only acceptable case: OutOfMemoryError for fallback behavior.

⌨ Example

```
// You CAN catch Error
try {
    throw new StackOverflowError();
} catch (StackOverflowError e) {
    System.out.println("Caught StackOverflowError"); // works
}

// You CAN catch Throwable (catches everything)
try {
    riskyMethod();
} catch (Throwable t) {
    System.out.println("Caught: " + t.getClass().getName());
}

// RARELY acceptable: OOM for graceful degradation
try {
    cache.loadAll(); // might OOM
} catch (OutOfMemoryError e) {
    cache.clear();   // free memory
    System.err.println("Cleared cache due to OOM");
}

// RULE: never catch Error in application code
// Let JVM crash and alert operations team
// Use JVM flags for heap dumps: -XX:+HeapDumpOnOutOfMemoryError
```

Q88. What is the output? What happens with static and instance initializers? [Tricky] [Static]

▶ Answer

Static initializers run once when the class is loaded, in textual order. Instance initializers run every time an object is created, before the constructor body (but after super()). Order: static init first (once), then for each new instance: super constructor, instance init blocks, constructor body.

⌨ Example

```
class Init {
    static int staticVal;
    int instanceVal;

    static {
        staticVal = 1;
        System.out.println("Static init: " + staticVal);
    }

    {
        instanceVal = 2;
        System.out.println("Instance init: " + instanceVal);
    }

    Init() {
        System.out.println("Constructor");
    }
```

```
}

System.out.println("Before first instantiation");
new Init();
System.out.println("---");
new Init();

// Output:
// Before first instantiation
// Static init: 1        <- class loaded here
// Instance init: 2
// Constructor
// ---
// Instance init: 2      <- no static init again
// Constructor
```

Q89. What is the output? String.valueOf(null) vs (String) null [Tricky] [Output]

Answer

String.valueOf(null) returns the String "null" (the 4-character string). Casting null to String gives null reference, which prints as "null" in println but is a null reference for other operations. String.valueOf((Object) null) is safe; String.valueOf(null) may cause a NullPointerException in some ambiguous overloads.

Example

```
String s1 = String.valueOf((Object) null); // "null" (4 chars)
String s2 = (String) null;                  // null reference

System.out.println(s1);         // null (prints 4-char string)
System.out.println(s2);         // null (prints "null" for null ref)
System.out.println(s1 == null); // false! s1 is "null" string
System.out.println(s2 == null); // true! s2 is null reference

System.out.println(s1.length()); // 4
try {
    System.out.println(s2.length()); // NullPointerException
} catch (NullPointerException e) { System.out.println("NPE"); }

// Ambiguous overload trap:
// String.valueOf has overloads for (Object), (char[]), etc.
// null can match char[] -> calls char[] overload -> NPE!
// String.valueOf(null)  // may choose char[] overload!
// Always cast: String.valueOf((Object) null)
```

Q90. What is the output? Generic method type inference edge cases. [Tricky] [Generics]

Answer

Java's type inference infers generic type parameters from context. When no context is available (e.g., returned from a method and immediately passed to println), inference may fall back to Object. Explicit type parameters (Collections.<String>emptyList()) override inference. In Java 8+, inference improved significantly with target typing.

Example

```
// Type inference from assignment context
List<String> list1 = Collections.emptyList(); // infers String

// Type inference from method parameter
void process(List<String> l) {}
process(Collections.emptyList()); // infers String
```

```
// Without context: infers Object
System.out.println(Collections.emptyList()); // []
// infers Object, but no observable difference

// Explicit type parameter
List<String> list2 = Collections.<String>emptyList();

// Method call chain: inference may fail
// getList().size(); // type of getList() determines element type

// Diamond inference (Java 7+)
Map<String, List<Integer>> map = new HashMap<>(); // infers

// Java 10+ var: type inference for local variables
var list3 = new ArrayList<String>(); // infers ArrayList<String>
var num = 42; // infers int
// var x; // COMPILE ERROR: cannot infer from nothing
```

Q91. What is the difference between CountDownLatch and CyclicBarrier? [Tricky] [Concurrency]

Answer

CountDownLatch: a one-time-use gate that opens when count reaches zero (countdown from N). One or many threads wait; others countDown. Not resettable. CyclicBarrier: reusable — all N parties must reach the barrier before any can proceed. After all arrive, barrier resets and can be reused. CyclicBarrier optionally runs a barrier action when tripped.

Example

```
// CountDownLatch: wait for N tasks to complete
CountDownLatch latch = new CountDownLatch(3);

// 3 worker threads
for (int i = 0; i < 3; i++) {
    new Thread(() -> {
        doWork();
        latch.countDown();  // decrement count
    }).start();
}

latch.await(); // waits until count = 0
System.out.println("All done");

// CyclicBarrier: all threads synchronize at the barrier
CyclicBarrier barrier = new CyclicBarrier(3, () ->
    System.out.println("Phase complete!")); // barrier action

for (int i = 0; i < 3; i++) {
    new Thread(() -> {
        doPhase1Work();
        barrier.await(); // wait for all 3
        doPhase2Work();  // start only after all reach barrier
        barrier.await(); // reusable!
    }).start();
}
```

Q92. What is the output? List<Integer>.subList() modification behavior. [Tricky] [Output]

Answer

subList() returns a view — not a copy — of the original list. Modifications to the sublist are reflected in the original list, and vice versa. Structural modifications to the original list (add/remove) after creating a sublist throw ConcurrentModificationException on any subsequent sublist operation.

⌨️ **Example**

```
List<Integer> list = new ArrayList<>(List.of(1,2,3,4,5));
List<Integer> sub = list.subList(1, 4);  // view of [2,3,4]

System.out.println(sub);  // [2, 3, 4]

// Modify sub: reflected in list
sub.set(0, 99);
System.out.println(list); // [1, 99, 3, 4, 5]

// Modify list structurally: ConcurrentModificationException
list.add(6);
try {
    System.out.println(sub.get(0)); // CME!
} catch (ConcurrentModificationException e) {
    System.out.println("CME after list modified");
}

// Use subList to remove a range efficiently
List<Integer> list2 = new ArrayList<>(List.of(1,2,3,4,5));
list2.subList(1, 4).clear();  // removes indices 1,2,3
System.out.println(list2);    // [1, 5]
```

Q93. What is a function composition with andThen vs compose? [Tricky] [Functional]

▶️ **Answer**

Function.andThen(g) creates f.andThen(g) = g(f(x)): apply f first, then g. Function.compose(g) creates f.compose(g) = f(g(x)): apply g first, then f. They are inverses: f.andThen(g) == g.compose(f). The difference is execution order — andThen is more intuitive (left-to-right), compose is mathematical notation.

⌨️ **Example**

```
Function<Integer, Integer> times2 = x -> x * 2;
Function<Integer, Integer> plus3  = x -> x + 3;

// andThen: times2 THEN plus3
Function<Integer, Integer> t2p3 = times2.andThen(plus3);
System.out.println(t2p3.apply(5)); // (5*2)+3 = 13

// compose: plus3 FIRST then times2
Function<Integer, Integer> p3t2 = times2.compose(plus3);
System.out.println(p3t2.apply(5)); // (5+3)*2 = 16

// Predicate composition
Predicate<Integer> pos  = n -> n > 0;
Predicate<Integer> even = n -> n % 2 == 0;

Predicate<Integer> posEven = pos.and(even);
Predicate<Integer> either  = pos.or(even);
Predicate<Integer> notPos  = pos.negate();

// Consumer composition: andThen (no compose)
Consumer<String> print  = System.out::println;
Consumer<String> log    = s -> log(s);
Consumer<String> both   = print.andThen(log);
```

Q94. What is the happens-before relationship in Java Memory Model? [Tricky] [Synchronization]

Answer

Happens-before (HB) defines which writes are visible to reads in a multithreaded program. Key HB rules: (1) Program order: within a thread, each action HB the next. (2) Monitor lock: unlock HB the next lock of the same monitor. (3) Volatile: write to volatile field HB all subsequent reads of that field. (4) Thread start: Thread.start() HB any action in the started thread. (5) Thread join: all actions in a thread HB join() returning.

Example

```
// Happens-before guarantees:

// 1. synchronized: unlock HB next lock
int x = 0;
synchronized(lock) { x = 42; }  // write
synchronized(lock) { int y = x; } // sees 42

// 2. volatile: write HB subsequent reads
volatile int flag = 0;
// Thread 1:
data = "hello";
flag = 1;           // volatile write
// Thread 2:
if (flag == 1) {    // volatile read
    print(data);    // guaranteed to see "hello"
}

// 3. Thread.start() HB actions in started thread
message = "ready";
thread.start();         // HB
// thread runs: print(message); // sees "ready"

// 4. Thread.join() HB caller actions after join
thread.join();
// caller sees all writes done by thread before join
```

Q95. What is the output of String.format with various format specifiers? [Tricky] [Output]

Answer

String.format provides printf-style formatting. Common specifiers: %s (toString), %d (integer), %f (float, 6 decimals by default), %n (newline), %% (literal %), %.2f (2 decimal places), %5d (right-padded to 5 chars), %-5d (left-padded). Forgetting the type can cause MissingFormatArgumentException or IllegalFormatConversionException.

Example

```
System.out.printf("%-10s | %5d | %.2f%n",
    "Alice", 25, 98765.432);
// Alice      |    25 | 98765.43

System.out.println(String.format("%,d", 1234567));  // 1,234,567
System.out.println(String.format("%05d", 42));       // 00042
System.out.println(String.format("%+d", 42));        // +42
System.out.println(String.format("%+d", -42));       // -42
System.out.println(String.format("%x", 255));        // ff
System.out.println(String.format("%X", 255));        // FF
System.out.println(String.format("%o", 8));          // 10
System.out.println(String.format("%e", 12345.678)); // 1.234568e+04

// IllegalFormatConversionException
try {
    String.format("%d", "hello"); // %d needs integer
} catch (java.util.IllegalFormatConversionException e) {
    System.out.println("Wrong type for %d");
```

```
}
```

Q96. What is the output? this() and super() call order in constructors. [Tricky] [Output]

▶ Answer

this() calls another constructor of the same class (constructor chaining). super() calls a superclass constructor. They are mutually exclusive as the first statement — you cannot use both. this() must also be the first statement. Constructor chaining via this() eventually calls super() (explicitly or implicitly). This prevents code duplication in multiple constructors.

⌨ Example

```
class Base {
    Base() { System.out.println("Base()"); }
    Base(int x) { System.out.println("Base(" + x + ")"); }
}

class Child extends Base {
    Child() {
        this(10);                    // 1: calls Child(int)
        System.out.println("Child()"); // 3
    }
    Child(int x) {
        super(x);                    // calls Base(int)
        System.out.println("Child(" + x + ")"); // 2
    }
}

new Child();
// Execution order:
// Child() -> this(10) -> Child(10) -> super(10) -> Base(10)
// Prints:
// Base(10)
// Child(10)
// Child()

// Cannot do: super() AND this() in same constructor
// Child() { super(); this(10); } // COMPILE ERROR
```

Q97. What is the diamond problem and how does Java resolve it? [Tricky] [Generics]

▶ Answer

The diamond problem occurs when a class inherits from two paths that both lead to the same supertype. Java does not allow multiple class inheritance, avoiding the classical diamond problem for classes. For interfaces with default methods, Java resolves by requiring explicit override when two interfaces provide conflicting defaults. Class implementation always wins over interface default.

⌨ Example

```
// Diamond via interfaces
interface A { default void hello() { System.out.println("A"); } }
interface B extends A { default void hello() { System.out.println("B"); } }
interface C extends A { default void hello() { System.out.println("C"); } }

// Rules for resolution:
// 1. Class always wins over interface
// 2. More specific interface wins (B and C win over A)
// 3. Conflict between B and C: must override explicitly

// Case: class wins over interface
class MyClass implements A {
```

```
    @Override public void hello() { System.out.println("Class"); }
}
interface A2 extends A {}
class D extends MyClass implements A2 {} // MyClass.hello() wins
new D().hello(); // "Class"

// Case: more specific interface wins
class E implements A, B {}    // B is more specific -> B.hello()
new E().hello(); // "B"

// Case: conflict requires explicit override
// class F implements B, C {} // COMPILE ERROR: must override
class F implements B, C {
    @Override public void hello() { B.super.hello(); }
}
```

Q98. What is the difference between findFirst() and findAny() in streams? [Tricky] [Streams]

Answer

findFirst() returns the first element in encounter order. findAny() may return any element — typically the first in sequential streams, but optimized for parallel streams (may return any element from any partition). Both return Optional. findAny() is preferred in parallel streams when you do not care which element is returned — it avoids the overhead of maintaining encounter order.

Example

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

// Sequential: both effectively the same
Optional<Integer> first = nums.stream()
    .filter(n -> n % 2 == 0)
    .findFirst();
System.out.println(first.get()); // always 2

Optional<Integer> any = nums.stream()
    .filter(n -> n % 2 == 0)
    .findAny();
System.out.println(any.get());   // 2 (sequential, usually first)

// Parallel: findAny() faster -- no order guarantee
Optional<Integer> parFirst = nums.parallelStream()
    .filter(n -> n % 2 == 0)
    .findFirst();
System.out.println(parFirst.get()); // always 2 (ordered)

Optional<Integer> parAny = nums.parallelStream()
    .filter(n -> n % 2 == 0)
    .findAny();
System.out.println(parAny.get()); // could be 2,4,6,8, or 10
```

Q99. What does Collections.sort() guarantee about equal elements (stability)? [Tricky] [Output]

Answer

Collections.sort() and List.sort() are guaranteed stable — equal elements maintain their original relative order. Arrays.sort() for objects is also stable (uses TimSort). Arrays.sort() for primitives is NOT stable (uses dual-pivot quicksort, which is not stable). This matters when sorting by one key and already sorted by another.

Example

```
record Person(String name, int age) {}
```

```
List<Person> people = new ArrayList<>(List.of(
    new Person("Alice", 30),
    new Person("Bob",   25),
    new Person("Carol", 30),
    new Person("Dave",  25)
));

// Sort by age -- equal ages maintain original order (stable)
people.sort(Comparator.comparingInt(Person::age));

// Bob and Dave both have age 25 -> Bob still before Dave
// Alice and Carol both have age 30 -> Alice still before Carol
people.forEach(p -> System.out.println(p.name()));
// Bob, Dave, Alice, Carol

// Primitives: Arrays.sort is NOT stable (quicksort)
int[] arr = {3, 1, 3, 2, 1};
Arrays.sort(arr); // stable vs unstable does not matter for primitives
// [1, 1, 2, 3, 3] -- ordering of equal ints cannot be observed
```

Q100. What is the output? What are the tricky aspects of Java's try-with-resources with inheritance? [Tricky] [Expert]

▶️ Answer

When a resource's close() calls another close() that throws, and the try body also throws, the body exception is the primary and close() exception is suppressed. When multiple resources are declared, they close in reverse order. A subclass close() that throws will be suppressed. Understanding this fully shows mastery of Java's exception hierarchy.

⌨️ Example

```
class Resource implements AutoCloseable {
    String name;
    Resource(String n) { name = n; System.out.println("Open: " + n); }
    public void close() {
        System.out.println("Close: " + name);
        if (name.equals("B")) throw new RuntimeException("Close B failed");
    }
}

try (
    Resource a = new Resource("A");
    Resource b = new Resource("B")  // closed first (reverse order)
) {
    System.out.println("Using resources");
    throw new RuntimeException("Body exception");
} catch (RuntimeException e) {
    System.out.println("Caught: " + e.getMessage());
    for (Throwable s : e.getSuppressed())
        System.out.println("Suppressed: " + s.getMessage());
}
// Output:
// Open: A
// Open: B
// Using resources
// Close: B          <- B closes first (reverse order)
// Close: A          <- A closes second
// Caught: Body exception   <- primary exception
// Suppressed: Close B failed  <- close exception is suppressed
```